BCIS Subscription Billing and Collection System: Technical Documentation

Deliverable D3. Requirements, architecture, modules, data model, API and IPC design, security model, deployment and backup/restore of the billing and collection system built for Bukidnon Cable and Internet Services (BCIS).

Related documents:

Document	Contents
`docs/erd.pdf`	Entity-relationship diagrams, one per module (also Appendix A)
Appendix B (`docs/generated/data-dictionary.md`)	Every table, column, constraint, index and trigger, generated from the database
Appendix C (`docs/generated/api-reference.md`)	Every API route with the permission it requires, and the role/permission matrix
`docs/deployment-guide.md`	Server and client installation, backups, restore, troubleshooting
`docs/security-review.md`	Security requirements and the evidence for each
`tests/acceptance-test-report.pdf`	Results of acceptance tests AT-01 to AT-12 and all automated tests
`docs/bug-log.md`, `docs/qa-checklist.md`	Defects found and fixed; final submission checklist

The appendices are generated from the running system (`pnpm --filter ./apps/api docs:reference`), so they always match the code.

---

1. Requirements

1.1 Business context

BCIS sells internet, cable TV and combined (combo) subscriptions in Bukidnon. Three office PCs handle subscribers, monthly billing, counter payments and GCash payments that customers report by message; field collectors go out on routes, collect cash, and remit it to the office. The business needs to know at any moment what each subscriber owes, which accounts are overdue, whether each collector's cash is complete, and who changed what.

The system replaces paper ledgers and spreadsheets. Its core promise is financial correctness: balances computed from posted records, history that is never overwritten, and numbers (invoices, receipts) that never repeat.

1.2 Users and roles

Role	Typical user	Main work
Owner / Super Admin	Business owner	Everything, including users, settings, backup and restore
Administrator	Office manager	Subscribers, plans, services, billing, payments, collections, reports
Cashier	Counter staff	Receive payments, issue receipts, record and verify GCash
Collection Supervisor	Collections lead	Areas, collectors, batches, remittances, reconciliation
Accounting / Auditor	Bookkeeper	Read everything financial, reports, audit trail
Technician	Field technician	Service status, suspensions and reconnections
Read-only Viewer	Management	Dashboard, reports and receivables

What each role may do is a set of permissions checked by the server (Appendix C has the matrix). A user may hold more than one role.

1.3 Functional requirements by module

Module	What the system does
Subscribers	Register subscribers with addresses and contacts; search by account number, name, phone, address, receipt, invoice or GCash reference; status changes (active, inactive, terminated, archived) with reasons; a nine-tab profile (overview, services, billing, payments, ledger, collection, history, documents, audit).
Plans and services	Plans per service type (internet, cable, combo) with price, speed or channels and fees; service accounts per subscriber with installation, activation, plan and rate changes, all kept as history.
Billing	Monthly billing in two steps (generate drafts, then finalize with numbers); installation and reconnection fees on the right invoice; voids with reason; credit and debit adjustments; the subscriber ledger and Statement of Account.
Payments	Counter payments (cash, bank transfer, cheque, other) with receipts; oldest-first allocation or manual allocation; advance payments kept as credit; reversals with reason.
GCash	Record customer-reported GCash payments with proof images; verify (posts the payment) or reject; duplicate references blocked.
Collections	Areas and collectors; collection batches built from what subscribers owe (route sheet); field collections; remittances; reconciliation of cash collected against cash remitted; shortages and overages recorded with a reason.
Receivables and service control	Outstanding and overdue lists; AR aging (current, 1-30, 31-60, 61-90, 90+ days); suspension candidates by configurable grace period and threshold; suspension with approval; reconnection workflow with fee or waiver.
Reports and dashboard	Dashboard KPIs and charts; collections, billing vs collection, revenue, AR aging, collector performance, subscriber master list, exceptions (adjustments and reversals), user activity, Statement of Account; PDF and XLSX export.
Administration	Users and roles, forced password change, account unlock; settings; audit log; backups, verification, test restore and restore; integrity check.

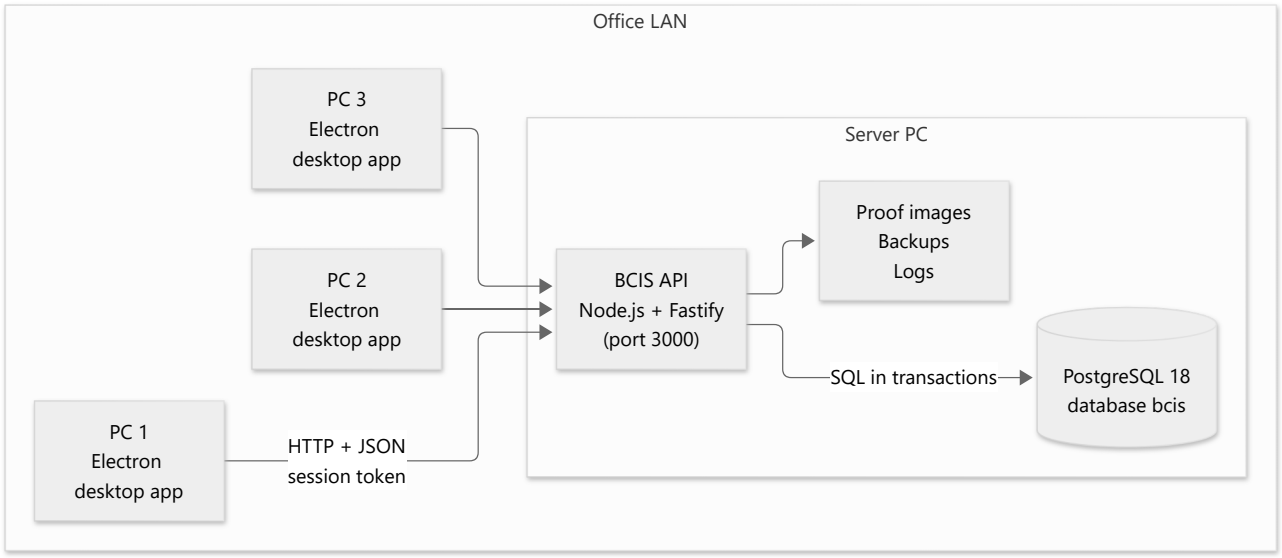
1.4 Non-functional requirements

Requirement	How it is met
Three PCs working at the same time	One API server; every change in a database transaction with row locks; numbering under locks (section 6). Proven by AT-09.
Financial correctness	Integer centavos; balances from the ledger; append-only history enforced by database triggers; reconciliation checks across reports (QA checklist item 12).
Security	Server-side authentication and permission checks on every route; argon2id passwords; hardened Electron client (section 9).
Auditability	Every change writes an audit row in the same transaction: who, what, when, why, old and new values.
Recoverability	Verified daily backups, test restores and a guarded restore (section 11).
Usability	Desktop UI with dense tables, keyboard search (Ctrl+K), right-aligned money in tabular numerals, status shown with text and colour, printable receipts and route sheets.

---

2. Architecture

2.1 Overview



The desktop app never talks to the database. Every PC sends requests to the one API, which checks the session and permissions, validates the input, runs the business rule in a database transaction, writes the audit row and answers. One of the three office PCs can be the server.

2.2 Why a client-server design with PostgreSQL

A shared database file (for example SQLite on a network folder) cannot safely take writes from three PCs at once: file locking over the network is unreliable, there is no server to enforce permissions, and each PC would need the business rules and full write access to the file. With PostgreSQL behind one API:

- transactions and row locks make concurrent posting safe (no duplicate receipt numbers, no lost updates);
- constraints, unique indexes and triggers enforce the most important rules in the database itself, whatever the client does;
- permissions are checked on the server, so a modified client cannot bypass them;
- backups are consistent snapshots (`pg_dump`) taken while people keep working.

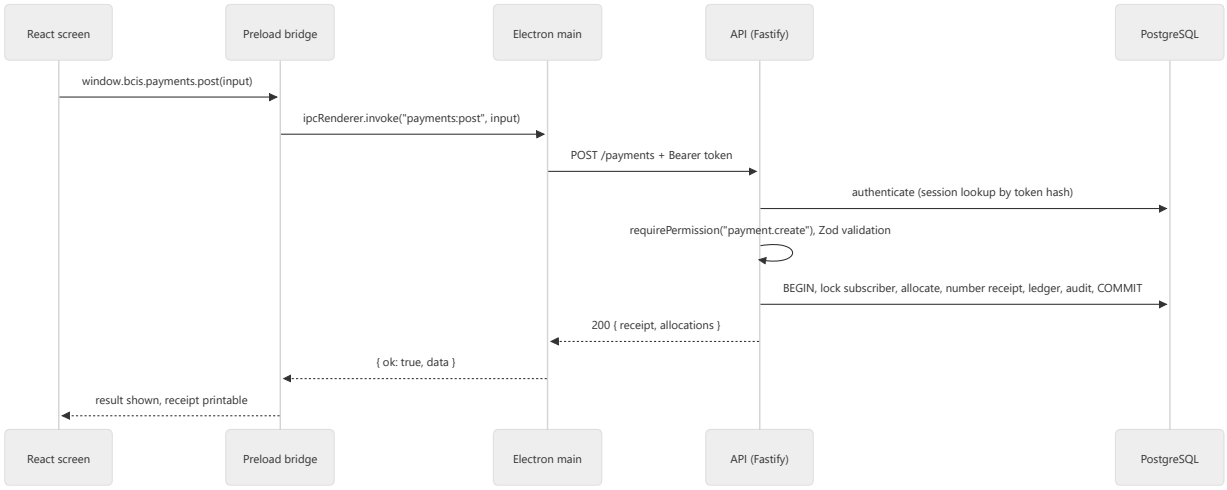
2.3 Technology stack

Layer	Technology
Language	TypeScript (strict), one pnpm monorepo
Server	Node.js 24, Fastify 5, Drizzle ORM, Zod 4, pino logging, argon2id (`@node-rs/argon2`)
Database	PostgreSQL 18 (migrations with drizzle-kit, plus hand-written triggers)
Desktop	Electron 44, React 19, Tailwind CSS 4, electron-vite, electron-builder (Windows installer)
Reports	pdfmake (PDF), ExcelJS (XLSX)
Tests	Vitest 5 against a real PostgreSQL test database
Quality	oxlint with type-aware rules; TypeScript typecheck on every package

2.4 Repository layout

Path	Contents
`apps/api`	The API server: `src/<module>/` (service, routes, tests per module), `scripts/` (build, reports, docs)
`apps/desktop`	The Electron app: `src/main` (window, IPC, session token), `src/preload` (the bridge and DTO types), `src/renderer` (React screens)
`packages/shared`	`@bcis/shared`: Zod schemas, money helpers, domain rules (allocation, aging, billing periods), permissions and navigation, used by both API and desktop
`database/migrations`	27 SQL migrations and the drizzle journal
`database/seeds`	How to seed (the seed scripts are in `apps/api/src/db`)
`deploy`	PowerShell scripts that install the server tasks and firewall rule
`docs`, `tests`	Documentation, acceptance report, screenshots

2.5 Life of a request



If anything fails before `COMMIT`, PostgreSQL rolls the whole transaction back: there is no half-posted payment, and the receipt number taken inside the transaction is returned to the counter. The client shows the error with a reference that matches the server log.

---

3. Module description

3.1 API modules (`apps/api/src`)

Module	Responsibility	Main tables
`auth`	Sign-in, sessions, lockout, `authenticate` and permission guards	users, sessions
`admin`	User management, roles, password reset and change	users, user_roles
`audit`	Writing audit rows (`writeAudit`) and the audit log queries	audit_logs
`subscribers`	Subscribers, addresses, contacts, status, search, profile data	subscribers, subscriber_addresses, subscriber_contacts
`plans`	Service plans	service_plans, service_types

Module	Responsibility	Main tables
`service-accounts`	Service accounts, status, plan, rate and collector changes, history	service_accounts, service_events
`billing`	Billing runs, invoices, voids, adjustments, ledger, Statement of Account data	billing_cycles, invoices, invoice_items, adjustments, ledger_entries
`payments`	Payments, allocation, credit, reversals, GCash submissions, proof storage	payments, payment_allocations, payment_reversals, gcash_submissions, payment_proofs
`collection`	Areas, collectors, batches, field collections, remittances, reconciliation, collector report	collection_areas, collectors, collector_assignments, collection_batches, batch_accounts, collector_remittances
`receivables`	Outstanding/overdue lists, aging, suspension candidates, suspensions, reconnections, settings	suspension_records, reconnection_records, application_settings
`reports`	Report queries, dashboard, PDF and XLSX rendering, audited exports	(reads all)
`backup`	Backups, manifest, verification, integrity check, test restore, restore	backup_history, integrity_checks
`http`	Error handling, request ids, logging, maintenance gate	
`db`	Connection, schema, migrations, document numbers, seeds	document_sequences

Each module follows the same shape: a **service** (business rules, transactions, audit), **routes** (HTTP, authentication, permission, validation, error mapping) and **tests**. Services throw typed errors (`PaymentError`, `BillingError`, ...) with a code and HTTP status; routes map them.

3.2 Desktop app (`apps/desktop`)

- **Main process** (`src/main/index.ts`): creates the window, holds the session token, and has one IPC handler per API call (122 handlers). It also saves exported files through the Save dialog and stores the server address in the user's settings folder.
- **Preload** (`src/preload/index.ts`): exposes `window.bcis` to the page through Electron's context bridge, with typed functions and the data types (DTOs) of every response.
- **Renderer** (`src/renderer/src`): React screens grouped like the menu: subscribers, billing, payments, collection, receivables, reports, dashboard, admin, plus shared UI parts (data table, money and date fields, badges, pager). The theme lives in one stylesheet.

3.3 Shared package (`packages/shared`)

Code both sides must agree on: every Zod input schema (the API validates with it, the forms use it for instant feedback), money parsing and formatting in centavos, the allocation algorithm, billing period arithmetic, aging buckets, cash variance, permission codes, role definitions and the navigation tree. 209 unit tests cover it.

---

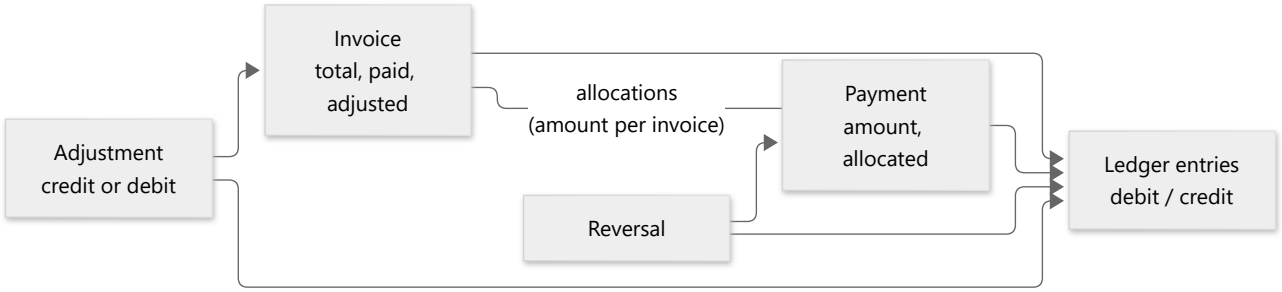
4. Data design

4.1 Principles

1. **Money is integer centavos** (`*_centavos` columns, integer types). Pesos are converted only at the form edge and for display. No floating point touches an authoritative amount.
2. **Posted financial records are never deleted or overwritten.** Payments are reversed, invoices voided, adjustments countered by an opposite adjustment. Allocations, reversals, ledger entries, proofs, service events, integrity checks and audit rows are append-only; database triggers reject `UPDATE` and `DELETE` on them.

3. **Master data is deactivated, not deleted**: users, plans, areas, collectors, addresses and contacts have an active flag; subscribers have a status.
4. **The ledger is the source of the balance.** Every invoice, payment, reversal, void and adjustment writes a ledger entry; a subscriber's balance is the sum of debits minus credits.
5. **Rules are enforced twice**: in the service, and in the database where it can be expressed (unique indexes, check constraints, triggers), so a bug or a direct SQL mistake cannot break them silently.
6. **Every schema change is a migration** (`database/migrations`), applied in order by `db:migrate`.

4.2 The financial model



- An **invoice** stores its total, the paid amount and the net adjustments. Its balance is `total + adjusted - paid`; its status is unpaid, partially paid, paid or void (and shown as overdue when open past its due date).
- A **payment** stores its amount and how much of it is allocated. **Allocations** say how much of the payment went to which invoice. A **partial payment** is simply an allocation smaller than the invoice balance: the invoice becomes partially paid and keeps the rest as its balance (AT-02).
- Money not allocated is the subscriber's **credit**: `amount - allocated` on posted payments. It is applied oldest-first when new invoices are finalized (AT-03).
- The **ledger** records every movement, so the balance (`sum(debit) - sum(credit)`) always equals open invoice balances minus credit. The integrity check and the reconciliation tests verify this identity across the whole database.

4.3 Entity-relationship diagram and data dictionary

The database has 36 tables in seven modules (access and audit; subscribers and services; billing and ledger; payments and GCash; collections; service control; settings, backups and integrity). The diagrams are in Appendix A and `docs/erd.pdf`; every column, constraint, index and trigger is in Appendix B.

4.4 Document numbers

Invoice (`INV-`), receipt (`RCPT-`), adjustment (`ADJ-`), collection batch (`CB-`) and backup (`BK-`) numbers come from `document_sequences`. A number is taken by updating the counter row inside the posting transaction, which locks the row until commit. Two PCs posting at once therefore wait for each other briefly and get consecutive numbers; a failed posting rolls the counter back, so numbers neither repeat nor skip. Voided invoices and reversed receipts keep their numbers forever.

---

5. Business rules

5.1 Billing

- **Generate** creates draft invoices for every active service account whose billing has started and that has no live invoice for the month: the subscription at the account's current rate, the installation fee in the month billing started, and any completed reconnection fee not yet billed. Drafts have no number and are not on the ledger. Running it again only adds drafts for accounts that became billable since (AT-11). Billing can run up to one month ahead.

- **Finalize** numbers the drafts, posts a ledger debit for each on its invoice date, and applies any available credit. Drafts of accounts suspended or terminated since generating are skipped.
- One live invoice per service account per month is also enforced by the unique index `invoices_one_per_period_idx`.
- **Void** (with a reason) is allowed only when no payment is applied; it posts a ledger credit and frees the month to be billed again.
- **Adjustments** are credits (discount, outage rebate, billing error, goodwill) or debits (penalty, reconnection fee, billing error) with a reason. A credit cannot exceed what is owed.

5.2 Payments and allocation

- A payment is applied to open invoices **oldest first** (by due date) unless a user with `payment.allocate` chooses the invoices by hand. Any amount left over becomes credit.
- A payment date may be in the past (late entry) but never in the future.
- **Reversal** (`payment.reverse`, reason required): the payment keeps its receipt number and is marked reversed; a reversal record and a ledger debit are added; the amounts it paid are taken off the invoices, which reopen, while its allocation rows stay as history. Nothing is deleted (AT-06).

5.3 GCash

Customers pay by GCash and send a screenshot. Staff record a submission (reference, sender, amount, date) and attach the proof image. A reviewer with `gcash.verify` then verifies it, which posts a GCash payment dated on the transaction date, or rejects it with a reason. Proof is required before verifying. A GCash reference can be used once: references are normalized (spaces removed, upper case) and protected by a unique index, so a second submission with the same reference is refused, naming the receipt that already used it (AT-05).

5.4 Collections

A batch moves through **open** → **in progress (dispatched)** → **submitted** → **remitted** → **reconciled** → **closed**, or is cancelled with a reason. It is built from the collector's subscribers who owe something, with a snapshot of what each owed (the route sheet). Field collections are payments posted through the batch, with receipts. Remittances record the cash handed over. Reconciliation compares cash collected with cash remitted: the difference is computed by the server, a shortage or overage needs a reason, and the reconciler must confirm the figure shown. Closing requires confirming the same difference again, so a short batch can never be closed as balanced (AT-07, AT-08). Reconciling and closing are separate permissions.

5.5 Receivables and service control

- **Aging** puts each open invoice in a bucket by days past its due date: current, 1-30, 31-60, 61-90, 90+. Unapplied credit is shown separately, so the aging total equals open invoice balances and the net equals the ledgers.
- **Suspension candidates** are active services with at least *threshold* invoices more than *grace period* days past due (both in Settings). The list is advice; suspending is a separate action with a reason and approver.
- **Reconnection**: request (with the fee, or a waiver with reason), assign a technician, complete. The fee goes on the next invoice.

---

6. Concurrency and consistency

- **One transaction per operation.** Each service function does its reads, writes, ledger entries, numbering and audit row in one transaction (`db.transaction`), passing the same handle to `writeAudit`.
- **Row locks** (`SELECT ... FOR UPDATE`) on the rows being changed. Money operations lock the subscriber first, then their invoices, always in that order, so two PCs working on the same subscriber queue instead of deadlocking.
- **Counter rows** for document numbers are locked until commit (section 4.4).
- **Advisory locks** guard the few rules that span many rows: "at least one owner remains", and "a subscriber is on at most one live collection batch".

- **Unique indexes** are the last line: one invoice per account per month, one GCash reference, unique document numbers.
- **What if the API crashes halfway through posting a payment?** The transaction never commits, so PostgreSQL discards all of it: no payment, no allocation, no ledger entry, no audit row, no receipt number used. The cashier sees an error and posts again. An unexpected error answers "check before retrying" with a reference, because a commit may have succeeded just before a network failure; the payment list shows the truth.

AT-09 posts payments from three sessions in parallel and checks for gaps, repeats and balance errors; `concurrency/at09.test.ts` does the same with billing, field collections and adjustments.

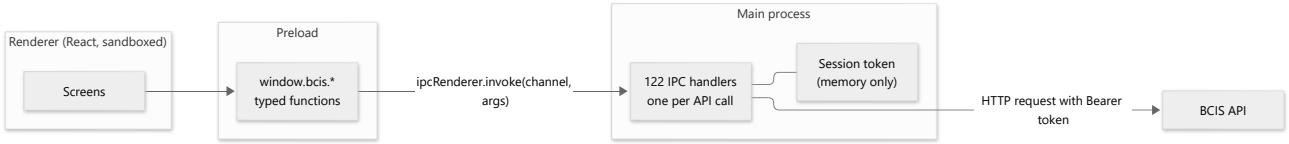
---

7. API design

- **Style**: JSON over HTTP, resource paths (`/subscribers/:id/ledger`), `GET` to read, `POST` for actions (each business action has its own endpoint and audit action, for example `/payments/:id/reverse`, `/subscribers/:id/status`), `PATCH` for edits of master data.
- **Authentication**: `POST /auth/login` returns a session token; every other call sends `Authorization: Bearer <token>`.
- **Authorization**: `authenticate` then `requirePermission(...)` on every route. The permission required by each route is listed in Appendix C, which is generated from the route table itself. A test calls every route without a token (401) and as a user with no roles (403).
- **Validation**: every body, query and path parameter is parsed with a Zod schema from `@bcis/shared`. Update schemas are strict (unknown fields refused) and need at least one change besides the reason. Failures answer `400 { error: "VALIDATION", issues: [{ path, message }] }`, which the forms map to fields.
- **Errors**: `{ error: CODE, message }` with the matching status: 401, 403, 404, 409 (state conflict, duplicate), 422 (business rule), 423 (account locked), 503 (database unavailable or maintenance). Unexpected errors answer 500 with a reference id that appears in the server log.
- **Lists** are paged (`page`, `pageSize`) and return `total`.
- **Exports**: `.../export?format=pdf|xlsx` returns the file; every export is audited.

---

8. IPC design (desktop)



- `contextIsolation` is on and `nodeIntegration` off: page code cannot reach Node.js or Electron.
- The renderer sees only `window.bcis`, a fixed set of functions (`payments.post`, `subscribers.get`, ...). There is no generic "call any URL" channel.
- The **session token stays in the main process** and is never given to the page.
- Each handler encodes ids with `encodeURIComponent`, calls the API and returns `{ ok: true, data }` or `{ ok: false, code, message, issues, reference }`. `UNAUTHENTICATED` sends the user back to sign-in.
- Exports are saved through the system Save dialog by the main process.
- The server address is set on the sign-in page; the main process checks that the address answers as a BCIS server before saving it.

---

9. Security model

Summary; the requirement-by-requirement evidence is in `docs/security-review.md`.

Area	Design
Passwords	argon2id hashes; at least 10 characters; new users must choose their own password at first sign-in; never logged (audit and log redaction of password and token fields).
Sign-in	5 wrong passwords lock the account for 15 minutes (also on "Change my password"); an owner can unlock.
Sessions	Random 32-byte token; only its SHA-256 hash is stored. Ends after 30 minutes idle or 12 hours. Sign-out, password change, password reset and restore end sessions.
Authorization	Permissions are read from the database on every request and checked on the server; hiding a menu item is only a convenience. Every route is tested for 401/403.
Input	Zod validation of every input; parameterized SQL only.
Desktop	Context isolation, no Node in the page, a strict Content Security Policy, all permission prompts (camera, location, ...) refused, no developer menu in the installed app.
Files	Proof images checked by content (PNG, JPEG, WebP), up to 5 MB, stored under a generated name in one folder, with a SHA-256 checked by the integrity check.
Audit	Append-only audit rows for every change, with actor, reason, old and new values.
Transport and data at rest	Plain HTTP inside the office LAN (accepted risk, documented); secrets only in the server's `.env`; backups stored on the server.

---

10. Reports, exports and logging

- Reports are computed from the same tables as the screens: collections by period and method, billing vs collection, revenue, AR aging, collector performance, subscriber master list, exceptions (adjustments and reversals), user activity, and the Statement of Account. The reconciliation test checks their totals against the underlying records and the exported XLSX against the report.
- Each report renders to PDF (pdfmake) and XLSX (ExcelJS) from one document description, so both formats show the same figures. Exports are audited (`report.export`).
- The server logs with pino to the console and to daily files (`logs/api.YYYY-MM-DD.N.log`, 30 days kept). Every response carries an `x-request-id`; an unexpected error shows that id to the user as a reference.

---

11. Deployment and backup/restore

Full steps are in `docs/deployment-guide.md`. In short:

- **Server PC**: PostgreSQL 18, Node 24 and the built API (`pnpm --filter ./apps/api build`). `deploy/install-server.ps1` registers two scheduled tasks (the API at start-up, kept running by a watchdog script; a daily backup) and a firewall rule for the local subnet.
- **Client PCs**: the Windows installer (`BCIS-Billing-Setup-x.y.z.exe`); the server address is entered once on the sign-in page.
- **Backups**: `pg_dump` (custom format) from one consistent snapshot while people keep working, plus a copy of the proof images and a `manifest.json` with row counts and a SHA-256 for every file. Each backup is verified (checksums, and `pg_restore --list` must read it). A daily backup runs automatically; the app warns when the last good backup is older than 24 hours.
- **Test restore** restores a backup into a scratch database (`bcis_verify`) and runs the integrity check on it, without touching live data.
- **Restore** (owner only, `backup.restore`): the user types the backup number and a reason; the API blocks all other requests (maintenance mode), takes a safety backup, restores with `pg_restore --clean --single-transaction` (all or nothing), keeps the backup history, signs everyone out, copies the proof images back and runs the integrity check (AT-12).
- **Integrity check** (also on demand): paid amounts equal allocations, allocations never exceed payments, ledger balances reconcile, counters are ahead of every issued number, proof files match their checksums, the migration count matches, and (after a restore) row counts match the backup.

12. Testing

Level	What	Where
Unit	Money, allocation, aging, billing periods, validation schemas	`packages/shared/src/*.test.ts` (209 tests)
Integration	Every service and route against PostgreSQL, including rollback on failure and audit rows	`apps/api/src/**/*.test.ts`
Acceptance	AT-01 to AT-12 with the spec's data, over HTTP	`apps/api/src/acceptance/acceptance.test.ts`
Reconciliation	Every money report against the records on the full demo dataset	`apps/api/src/acceptance/reconciliation.test.ts`
Security	Every route refuses missing sessions and missing permissions	`apps/api/src/security/route-guards.test.ts`
Concurrency	Parallel billing, payments, collections; three-PC script	`apps/api/src/concurrency/`
Desktop	Manual walkthrough of every workflow; screenshots	`tests/screenshots/`

Results: `tests/acceptance-test-report.pdf`. Defects: `docs/bug-log.md`.

---

13. Design decisions

Decision	Alternatives	Why
One API server and PostgreSQL	Shared SQLite or Access file	Safe concurrent writes, server-side permissions, constraints and consistent backups (section 2.2).
Integer centavos	Decimal or floating point	Exact arithmetic everywhere, including JavaScript, with no rounding drift.
Reverse, never delete	Edit or delete wrong postings	The history must explain every balance; auditors see the mistake and its correction.
Ledger plus stored invoice totals	Compute everything from payments	Fast balances and statements; the integrity check proves both agree.
Two-step billing (draft, finalize)	Create final invoices directly	The month can be reviewed and fixed before numbers are issued; re-running is safe.
Oldest-first allocation, leftover as credit	Pay the newest bill, or refuse overpayment	Matches how arrears are collected; advance payments are kept, never lost.
GCash reference unique	Warn only	A reference is one real transfer; posting it twice would invent money.
Server computes the batch difference; client confirms it	Trust the figure typed	Nobody can reconcile on figures they did not see; a shortage cannot be closed as balanced.
Business rules in services and `@bcis/shared`	Rules in the React screens	One authority for every client; screens cannot bypass rules.
One IPC handler per call; token in main	Generic proxy; token in the page	A compromised page cannot call arbitrary endpoints or steal the session.
Rules also in the database (triggers, unique indexes)	Application checks only	A bug or a manual SQL change cannot silently corrupt financial history.

---

14. Limitations and future work

- **Plain HTTP on the LAN.** Acceptable inside the office network; a reverse proxy with TLS would be needed before any access from outside.
- **No partial-month (pro-rated) billing.** A service reconnected mid-month is billed from the next month.
- **Collector mobile app (PWA).** Collectors use paper route sheets today. A mobile app would need: collector accounts with a permission limited to their own batch; the API reachable from outside (TLS, rate limiting); an offline queue of collections with a client-generated id per collection, so a retry after a lost connection cannot post twice (idempotency key); receipts numbered on sync by the server (or a per-collector series); and conflict rules when the office posts a payment for the same subscriber before the collector syncs.
- **Notifications** (SMS or email reminders) and a customer self-service portal are not built.

Appendices

- **Appendix A**: Entity-relationship diagrams
- **Appendix B**: Data dictionary
- **Appendix C**: API reference and role/permission matrix

All three follow, generated from the running system.

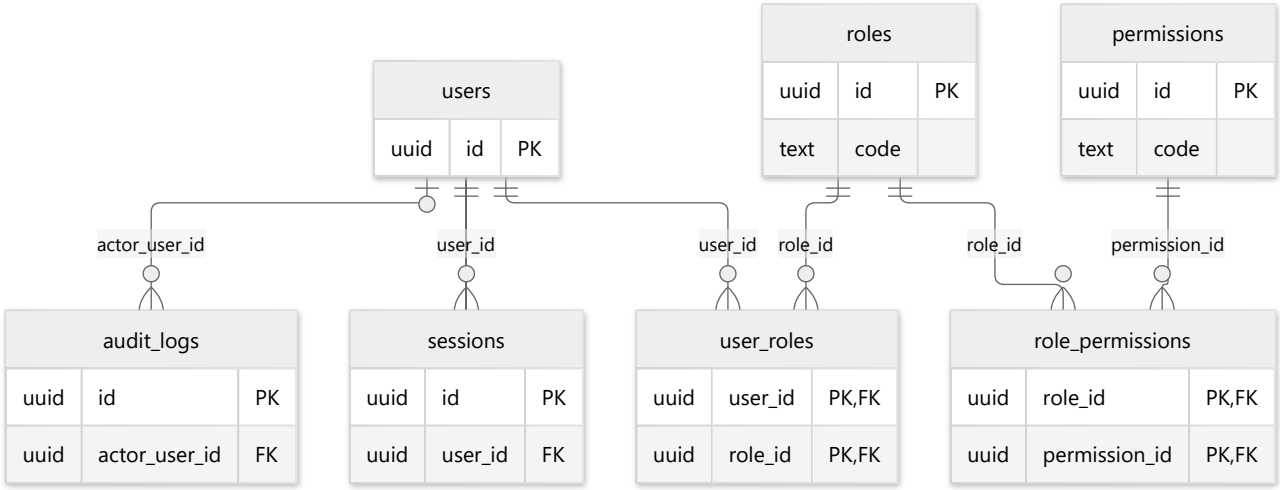
Entity-Relationship Diagram

Generated from the migrated database. One diagram per module; tables from other modules appear without columns where they are referenced. Only keys, money, document numbers, statuses and key dates are drawn; every column is in the data dictionary. Most tables also record who created or changed a row (a foreign key to `users`); those links are drawn only in the first diagram, to keep the others readable.

Notation: `||--o{` one to many; `|o--o{` the parent is optional (nullable foreign key).

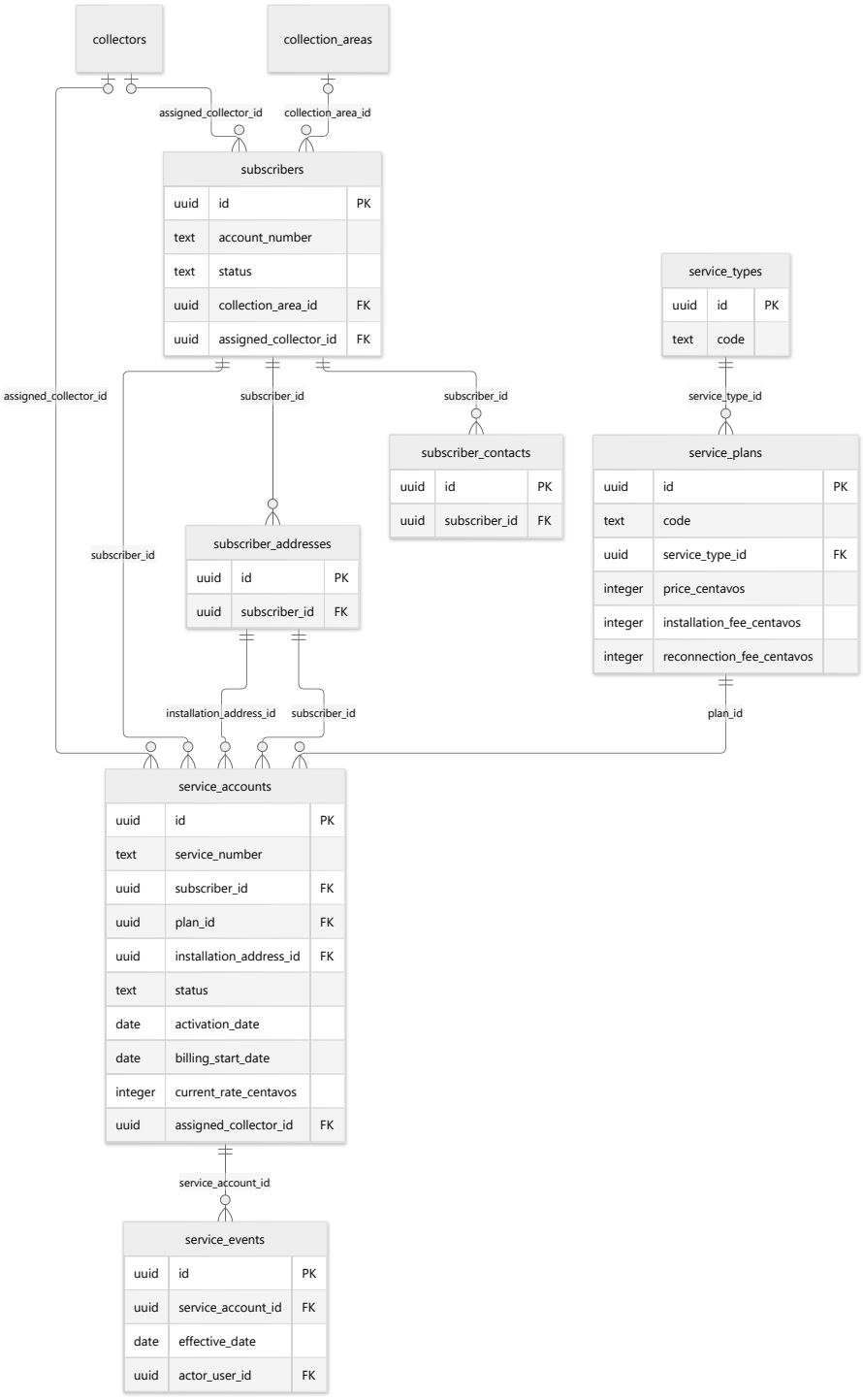
Access and audit

Who can sign in, what each role may do, live sessions, and the append-only audit trail.



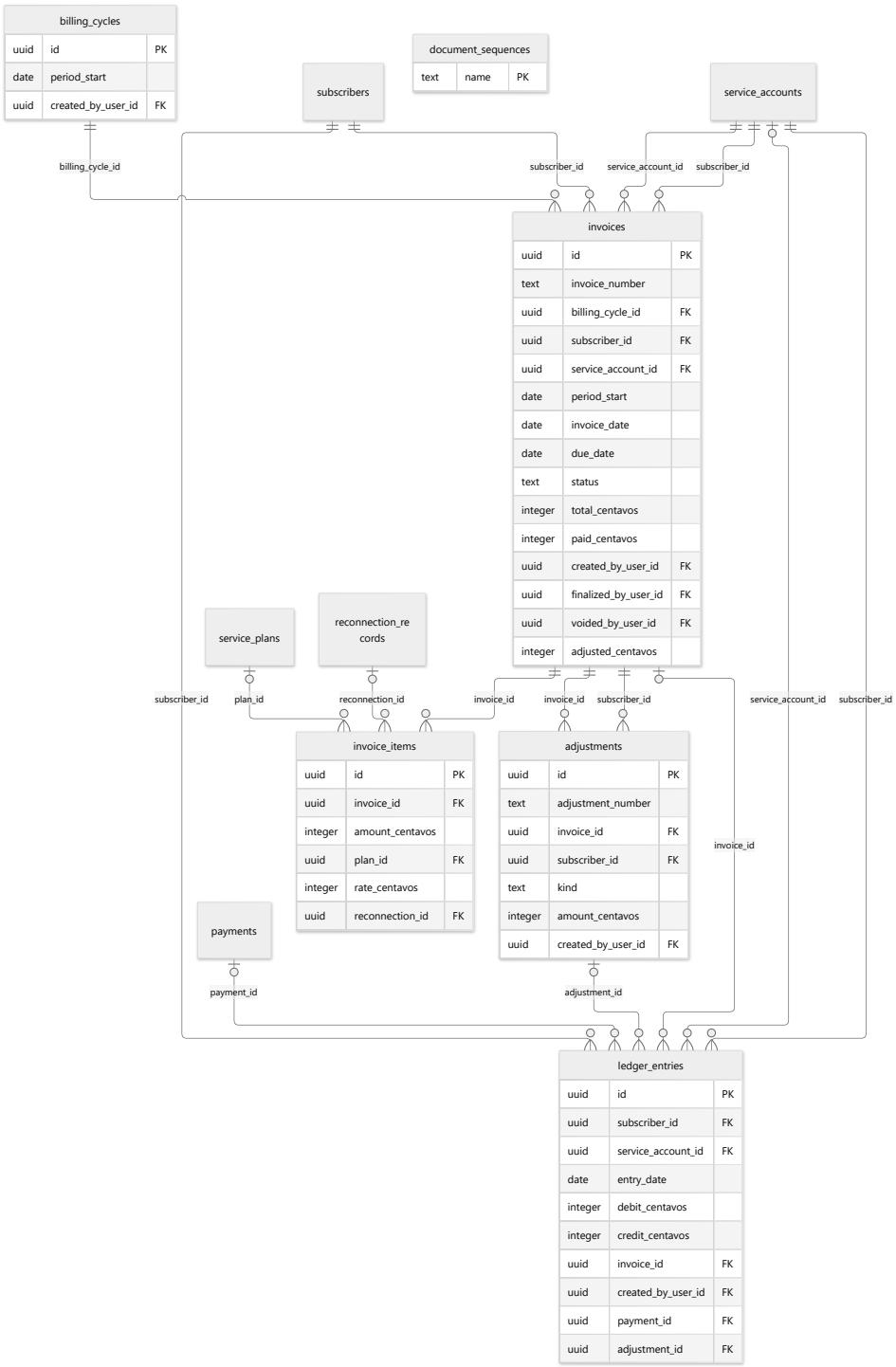
Subscribers and services

Customers, their addresses and contacts, the plans offered, and each service account with its history.



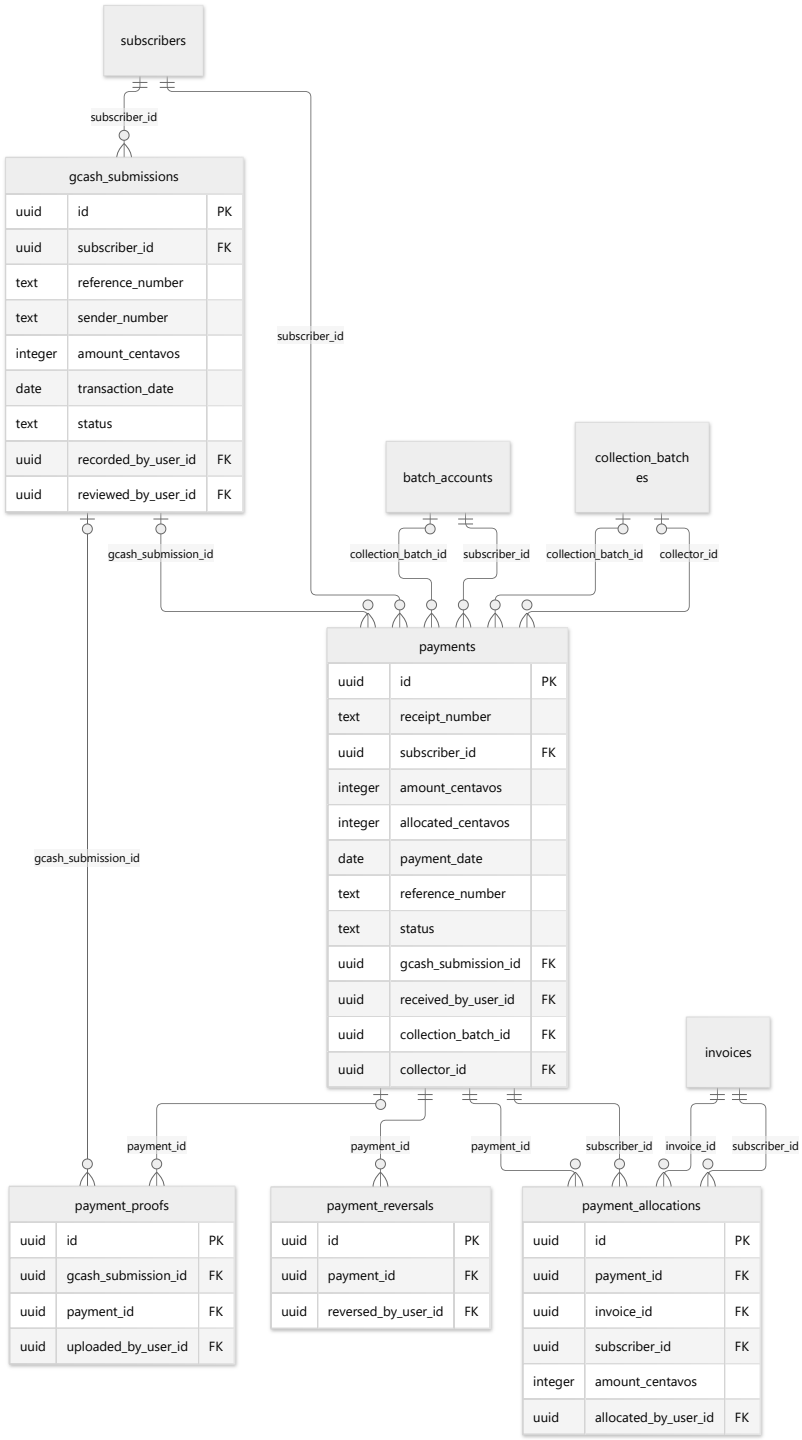
Billing and ledger

Monthly billing runs, invoices and their lines, adjustments, and the subscriber ledger that every balance comes from.



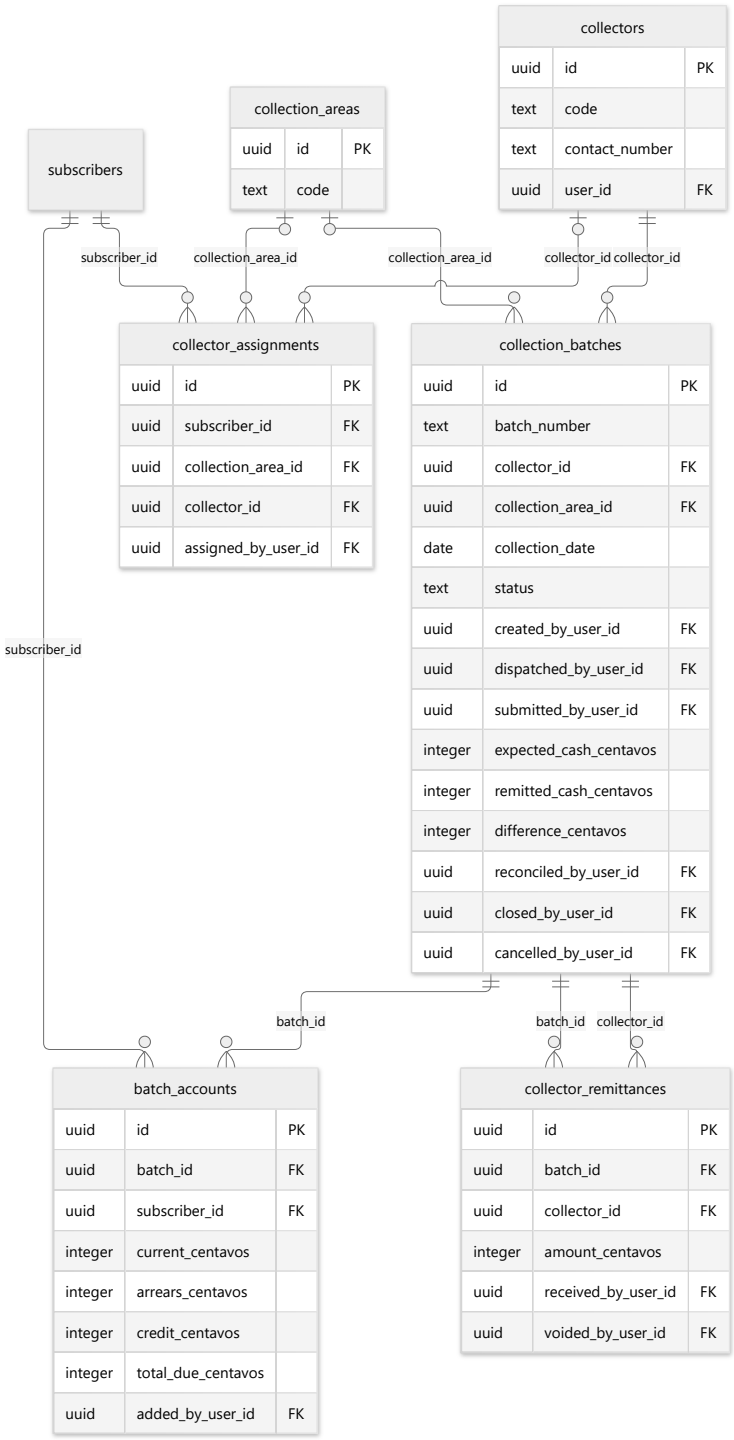
Payments and GCash

Money received, how it was applied to invoices, reversals, and GCash submissions with their proof images.



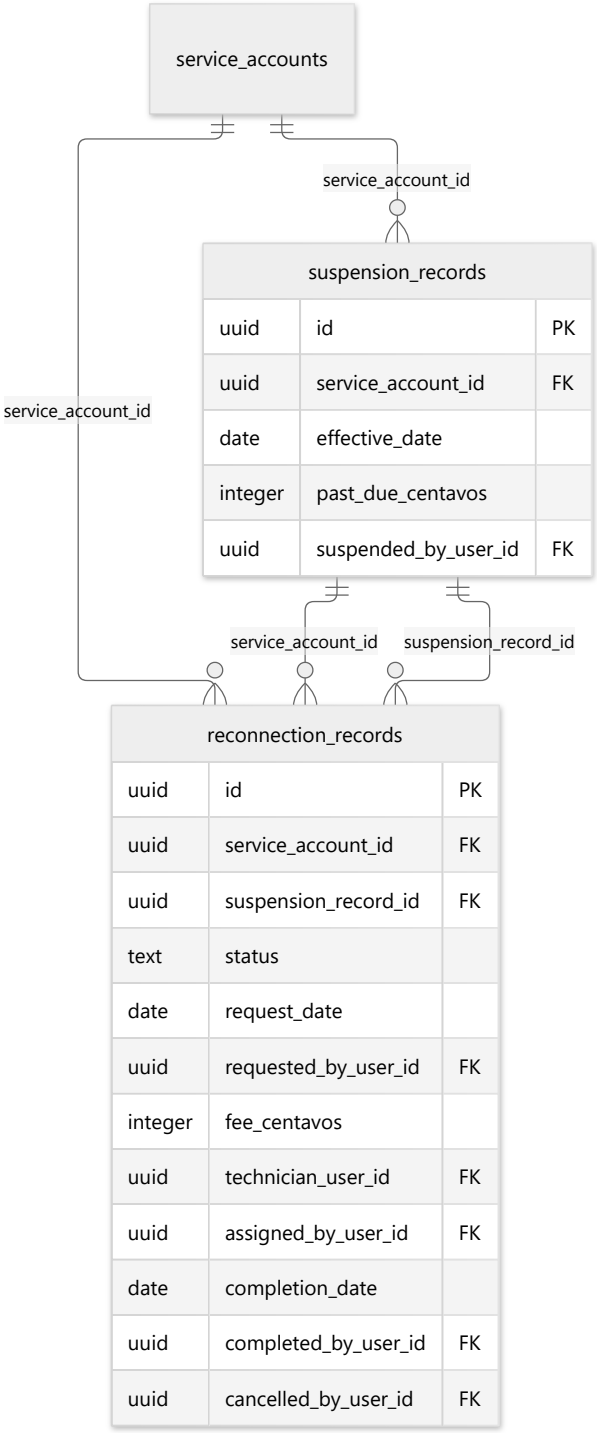
Collections

Collection areas, collectors, their batches (route sheets), field collections and cash remittances.



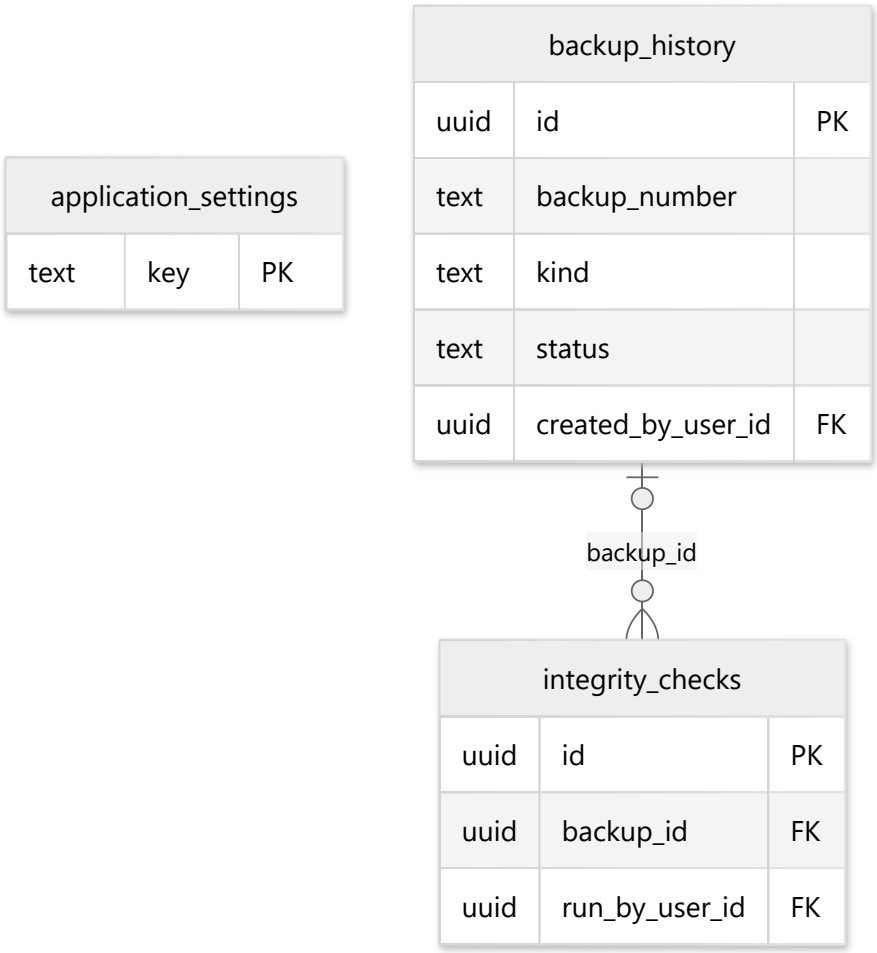
Service control

Suspensions for non-payment and the reconnection workflow, including the reconnection fee.



Settings, backups and integrity

Business settings, the backup history and the results of integrity checks.



Data Dictionary

Generated from the migrated database: 36 tables in 7 modules. Money is stored as integer centavos (`*_centavos`); dates as `date`, moments as `timestamptz`. Checks and triggers listed here are enforced by PostgreSQL itself, whatever the application does.

Access and audit

Who can sign in, what each role may do, live sessions, and the append-only audit trail.

users

Staff accounts. Password stored as an argon2id hash; failed attempts and lockout tracked here. Deactivated, never deleted.

Column	Type	Null	Default	Key
id	uuid	no	`gen_random_uuid()`	PK
username	text	no		
full_name	text	no		
password_hash	text	no		
is_active	boolean	no	`true`	
failed_login_attempts	integer	no	`0`	
locked_until	timestamptz	yes		
last_login_at	timestamptz	yes		
created_at	timestamptz	no	`now()`	
updated_at	timestamptz	no	`now()`	
must_change_password	boolean	no	`false`	

- Check `users_username_lowercase`: `CHECK ((username = lower(username)))`
- Unique `users_username_unique`: `UNIQUE (username)`

roles

The seven roles (owner, administrator, cashier, collection supervisor, auditor, technician, viewer).

Column	Type	Null	Default	Key
id	uuid	no	`gen_random_uuid()`	PK
code	text	no		
name	text	no		
description	text	no		

- Unique `roles_code_unique`: `UNIQUE (code)`

permissions

Permission codes checked by the API on every request.

Column	Type	Null	Default	Key
id	uuid	no	`gen_random_uuid()`	PK
code	text	no		
description	text	no		

- Unique `permissions_code_unique`: `UNIQUE (code)`

role_permissions

Which permissions each role grants.

Column	Type	Null	Default	Key
role_id	uuid	no		PK, FK → roles.id
permission_id	uuid	no		PK, FK → permissions.id

- Unique index `role_permissions_role_id_permission_id_pk`: `(role_id, permission_id)`

user_roles

Which roles each user holds.

Column	Type	Null	Default	Key
user_id	uuid	no		PK, FK → users.id
role_id	uuid	no		PK, FK → roles.id

- Unique index `user_roles_user_id_role_id_pk`: `(user_id, role_id)`

sessions

Signed-in sessions. Only a SHA-256 hash of the session token is stored.

Column	Type	Null	Default	Key
id	uuid	no	`gen_random_uuid()`	PK
user_id	uuid	no		FK → users.id
token_hash	text	no		
created_at	timestamptz	no	`now()`	
last_activity_at	timestamptz	no	`now()`	
expires_at	timestamptz	no		
revoked_at	timestamptz	yes		

- Unique `sessions_token_hash_unique`: `UNIQUE (token_hash)`
- Index `sessions_user_idx`: `(user_id)`

audit_logs

Every change: actor, action, record, reason, old and new values. Append-only (database trigger).

Column	Type	Null	Default	Key
id	uuid	no	`gen_random_uuid()`	PK
occurred_at	timestamptz	no	`now()`	

Column	Type	Null	Default	Key
actor_user_id	uuid	yes		FK → users.id
action	text	no		
entity_type	text	no		
entity_id	text	yes		
reason	text	yes		
old_values	jsonb	yes		
new_values	jsonb	yes		

- Index `audit_logs_action_idx`: `(action)`
- Index `audit_logs_actor_idx`: `(actor_user_id)`
- Index `audit_logs_entity_idx`: `(entity_type, entity_id)`
- Index `audit_logs_occurred_idx`: `(occurred_at)`
- Trigger `audit_logs_no_update_delete` (BEFORE): `audit_logs_block_modification()`

Subscribers and services

Customers, their addresses and contacts, the plans offered, and each service account with its history.

subscribers

Customer accounts (BCIS- numbers). Status active, inactive, terminated or archived; never deleted.

Column	Type	Null	Default	Key
id	uuid	no	`gen_random_uuid()`	PK
account_number	text	no	`('BCIS-'::text || lpad((nextval('subscriber_account_seq'::`	
full_name	text	no		
status	text	no	`'active'::text`	
billing_day	integer	no		
collection_area_id	uuid	yes		FK → collection_areas.id
assigned_collector_id	uuid	yes		FK → collectors.id
notes	text	yes		
created_at	timestamptz	no	`now()`	
updated_at	timestamptz	no	`now()`	

- Unique `subscribers_account_number_unique`: `UNIQUE (account_number)`
- Check `subscribers_billing_day_valid`: `CHECK (((billing_day >= 1) AND (billing_day <= 28)))`
- Check `subscribers_status_valid`: `CHECK ((status = ANY (ARRAY['active'::text, 'inactive'::text, 'terminated'::text, 'archived'::text])))`
- Index `subscribers_account_number_trgm_idx`: `(account_number gin_trgm_ops)`
- Index `subscribers_area_idx`: `(collection_area_id)`
- Index `subscribers_collector_idx`: `(assigned_collector_id)`
- Index `subscribers_full_name_idx`: `(full_name)`
- Index `subscribers_full_name_trgm_idx`: `(full_name gin_trgm_ops)`
- Index `subscribers_status_idx`: `(status)`

subscriber_addresses

Addresses of a subscriber; at most one is primary. Deactivated, never deleted.

Column	Type	Null	Default	Key
id	uuid	no	`gen_random_uuid()`	PK
subscriber_id	uuid	no		FK → subscribers.id
label	text	yes		
line1	text	no		
barangay	text	no		
city	text	no		
province	text	yes		
landmark	text	yes		
is_primary	boolean	no	`false`	
is_active	boolean	no	`true`	
created_at	timestamptz	no	`now()`	
updated_at	timestamptz	no	`now()`	

- Unique index `subscriber_addresses_id_subscriber_idx`: `(id, subscriber_id)`
- Unique index `subscriber_addresses_one_primary_idx`: `(subscriber_id) WHERE is_primary`
- Index `subscriber_addresses_search_trgm_idx`: `((((((((((line1 || ' '::text) || barangay) || ' '::text) || city) || ' '::text) || COALESCE(province, ''::text)) || ' '::text) || COALESCE(landmark, ''::text))) gin_trgm_ops)`
- Index `subscriber_addresses_subscriber_idx`: `(subscriber_id)`

subscriber_contacts

Phone numbers and emails; at most one is primary. Searchable by digits (trigram index).

Column	Type	Null	Default	Key
id	uuid	no	`gen_random_uuid()`	PK
subscriber_id	uuid	no		FK → subscribers.id
type	text	no		
value	text	no		
contact_name	text	yes		
is_primary	boolean	no	`false`	
is_active	boolean	no	`true`	
created_at	timestamptz	no	`now()`	
updated_at	timestamptz	no	`now()`	

- Check `subscriber_contacts_type_valid`: `CHECK ((type = ANY (ARRAY['mobile'::text, 'landline'::text, 'email'::text, 'other'::text])))`
- Index `subscriber_contacts_digits_trgm_idx`: `(regexp_replace(value, '\D'::text, ''::text, 'g'::text) gin_trgm_ops)`
- Unique index `subscriber_contacts_one_primary_idx`: `(subscriber_id) WHERE is_primary`
- Index `subscriber_contacts_subscriber_idx`: `(subscriber_id)`
- Index `subscriber_contacts_value_idx`: `(value)`

- Index `subscriber_contacts_value_trgm_idx`: `(value gin_trgm_ops)`

service_types

Internet, cable, combo.

Column	Type	Null	Default	Key
id	uuid	no	`gen_random_uuid()`	PK
code	text	no		
name	text	no		

- Unique `service_types_code_unique`: `UNIQUE (code)`

service_plans

Plans with price, speed or channel count and installation fee. Deactivated, never deleted.

Column	Type	Null	Default	Key
id	uuid	no	`gen_random_uuid()`	PK
code	text	no		
name	text	no		
service_type_id	uuid	no		FK → service_types.id
price_centavos	integer	no		
installation_fee_centavos	integer	no	`0`	
reconnection_fee_centavos	integer	no	`0`	
description	text	yes		
speed_mbps	integer	yes		
channel_count	integer	yes		
is_active	boolean	no	`true`	
created_at	timestamptz	no	`now()`	
updated_at	timestamptz	no	`now()`	

- Check `service_plans_attributes_positive`: `CHECK ((((speed_mbps IS NULL) OR (speed_mbps > 0)) AND ((channel_count IS NULL) OR (channel_count > 0))))`
- Unique `service_plans_code_unique`: `UNIQUE (code)`
- Check `service_plans_fees_nonneg`: `CHECK (((installation_fee_centavos >= 0) AND (reconnection_fee_centavos >= 0)))`
- Check `service_plans_price_nonneg`: `CHECK ((price_centavos >= 0))`
- Index `service_plans_active_idx`: `(is_active)`
- Index `service_plans_type_idx`: `(service_type_id)`

service_accounts

One connection (SVC- number): plan, installation address, current rate, status and billing start.

Column	Type	Null	Default	Key
id	uuid	no	`gen_random_uuid()`	PK
service_number	text	no	`('SVC-'::text || lpad((nextval('service_account_seq'::regc`	

Column	Type	Null	Default	Key
subscriber_id	uuid	no		FK → subscriber_addresses.subscriber_id
plan_id	uuid	no		FK → service_plans.id
installation_address_id	uuid	no		FK → subscriber_addresses.id
status	text	no	`'pending'::text`	
activation_date	date	yes		
billing_start_date	date	yes		
billing_day	integer	no		
current_rate_centavos	integer	no		
assigned_collector_id	uuid	yes		FK → collectors.id
notes	text	yes		
created_at	timestamptz	no	`now()`	
updated_at	timestamptz	no	`now()`	

- Check `service_accounts_activated_valid`: `CHECK (((status = ANY (ARRAY['pending'::text, 'terminated'::text])) OR (activation_date IS NOT NULL)))`
- Check `service_accounts_billing_day_valid`: `CHECK (((billing_day >= 1) AND (billing_day <= 28)))`
- Check `service_accounts_dates_valid`: `CHECK ((((activation_date IS NULL) = (billing_start_date IS NULL)) AND ((billing_start_date IS NULL) OR (billing_start_date >= activation_date))))`
- Check `service_accounts_rate_nonneg`: `CHECK ((current_rate_centavos >= 0))`
- Unique `service_accounts_service_number_unique`: `UNIQUE (service_number)`
- Check `service_accounts_status_valid`: `CHECK ((status = ANY (ARRAY['pending'::text, 'active'::text, 'suspended'::text, 'terminated'::text])))`
- Index `service_accounts_collector_idx`: `(assigned_collector_id)`
- Unique index `service_accounts_id_subscriber_idx`: `(id, subscriber_id)`
- Index `service_accounts_plan_idx`: `(plan_id)`
- Index `service_accounts_service_number_trgm_idx`: `(service_number gin_trgm_ops)`
- Index `service_accounts_status_idx`: `(status)`
- Index `service_accounts_subscriber_idx`: `(subscriber_id)`

service_events

History of each service: activation, plan and rate changes, suspensions, reconnections. Append-only.

Column	Type	Null	Default	Key
id	uuid	no	`gen_random_uuid()`	PK
service_account_id	uuid	no		FK → service_accounts.id
event_type	text	no		
from_status	text	yes		
to_status	text	yes		
old_values	jsonb	yes		
new_values	jsonb	yes		
effective_date	date	no	`CURRENT_DATE`	
reason	text	yes		
actor_user_id	uuid	no		FK → users.id

Column	Type	Null	Default	Key
occurred_at	timestamptz	no	`now()`	

- Check `service_events_type_valid`: `CHECK ((event_type = ANY (ARRAY['created'::text, 'status_change'::text, 'rate_change'::text, 'plan_change'::text, 'collector_change'::text, 'update'::text, 'reconnection_request'::text, 'reconnection_assign'::text, 'reconnection_cancel'::text])))`
- Index `service_events_account_idx`: `(service_account_id, occurred_at)`
- Trigger `service_events_no_update_delete` (BEFORE): `service_events_block_modification()`

Billing and ledger

Monthly billing runs, invoices and their lines, adjustments, and the subscriber ledger that every balance comes from.

billing_cycles

One row per billed month; locked while a run generates or finalizes.

Column	Type	Null	Default	Key
id	uuid	no	`gen_random_uuid()`	PK
period_start	date	no		
period_end	date	no		
created_by_user_id	uuid	no		FK → users.id
created_at	timestamptz	no	`now()`	

- Check `billing_cycles_first_of_month`: `CHECK ((EXTRACT(day FROM period_start) = (1)::numeric))`
- Unique `billing_cycles_period_start_unique`: `UNIQUE (period_start)`
- Check `billing_cycles_whole_month`: `CHECK ((period_end = (((period_start + '1 mon'::interval) - '1 day'::interval))::date))`

invoices

One per service account per month (INV- numbers). Draft until finalized; paid and adjusted amounts kept on the row; voided, never deleted.

Column	Type	Null	Default	Key
id	uuid	no	`gen_random_uuid()`	PK
invoice_number	text	yes		
billing_cycle_id	uuid	no		FK → billing_cycles.id
subscriber_id	uuid	no		FK → service_accounts.subscriber_id
service_account_id	uuid	no		FK → service_accounts.id
period_start	date	no		
period_end	date	no		
invoice_date	date	no		
due_date	date	no		
status	text	no	`'draft'::text`	
total_centavos	integer	no		
paid_centavos	integer	no	`0`	
created_by_user_id	uuid	no		FK → users.id

Column	Type	Null	Default	Key
created_at	timestamptz	no	`now()`	
finalized_by_user_id	uuid	yes		FK → users.id
finalized_at	timestamptz	yes		
voided_by_user_id	uuid	yes		FK → users.id
voided_at	timestamptz	yes		
void_reason	text	yes		
adjusted_centavos	integer	no	`0`	

- Check `invoices_draft_shape`: `CHECK ((((status = 'draft'::text) = (invoice_number IS NULL)) AND ((status = 'draft'::text) = (finalized_at IS NULL))))`
- Check `invoices_draft_unpaid`: `CHECK (((status <> 'draft'::text) OR ((paid_centavos = 0) AND (adjusted_centavos = 0))))`
- Check `invoices_due_after_issue`: `CHECK ((due_date >= invoice_date))`
- Check `invoices_effective_total_nonneg`: `CHECK (((total_centavos + adjusted_centavos) >= 0))`
- Unique `invoices_invoice_number_unique`: `UNIQUE (invoice_number)`
- Check `invoices_paid_range`: `CHECK (((paid_centavos >= 0) AND (paid_centavos <= (total_centavos + adjusted_centavos))))`
- Check `invoices_paid_status_consistent`: `CHECK (((status <> ALL (ARRAY['unpaid'::text, 'partially_paid'::text, 'paid'::text, 'credited'::text])) OR (status = CASE WHEN (((total_centavos + adjusted_centavos) = 0) AND (adjusted_centavos < 0)) THEN 'credited'::text WHEN (paid_centavos = (total_centavos + adjusted_centavos)) THEN 'paid'::text WHEN (paid_centavos = 0) THEN 'unpaid'::text ELSE 'partially_paid'::text END)))`
- Check `invoices_period_valid`: `CHECK ((period_end >= period_start))`
- Check `invoices_status_valid`: `CHECK ((status = ANY (ARRAY['draft'::text, 'unpaid'::text, 'partially_paid'::text, 'paid'::text, 'void'::text, 'credited'::text])))`
- Check `invoices_total_nonneg`: `CHECK ((total_centavos >= 0))`
- Check `invoices_void_shape`: `CHECK ((((status = 'void'::text) = (voided_at IS NOT NULL)) AND ((status <> 'void'::text) OR (void_reason IS NOT NULL))))`
- Index `invoices_cycle_idx`: `(billing_cycle_id)`
- Index `invoices_due_date_idx`: `(due_date)`
- Unique index `invoices_id_subscriber_idx`: `(id, subscriber_id)`
- Index `invoices_invoice_number_trgm_idx`: `(invoice_number gin_trgm_ops)`
- Unique index `invoices_one_per_period_idx`: `(service_account_id, period_start) WHERE (status <> 'void'::text)`
- Index `invoices_status_idx`: `(status)`
- Index `invoices_subscriber_idx`: `(subscriber_id, period_start)`
- Trigger `invoices_protect_finalized` (BEFORE): `invoices_protect_finalized()`

invoice_items

Invoice lines: subscription, installation fee, reconnection fee.

Column	Type	Null	Default	Key
id	uuid	no	`gen_random_uuid()`	PK
invoice_id	uuid	no		FK → invoices.id
line_no	integer	no		
item_type	text	no		
description	text	no		
amount_centavos	integer	no		

Column	Type	Null	Default	Key
plan_id	uuid	yes		FK → service_plans.id
rate_centavos	integer	yes		
reconnection_id	uuid	yes		FK → reconnection_records.id

- Check `invoice_items_line_positive`: `CHECK ((line_no >= 1))`
- Check `invoice_items_reconnection_fee_only`: `CHECK (((reconnection_id IS NULL) OR (item_type = 'reconnection_fee'::text)))`
- Check `invoice_items_type_valid`: `CHECK ((item_type = ANY (ARRAY['subscription'::text, 'installation_fee'::text, 'reconnection_fee'::text, 'discount'::text, 'penalty'::text, 'adjustment'::text])))`
- Unique index `invoice_items_line_idx`: `(invoice_id, line_no)`
- Index `invoice_items_reconnection_idx`: `(reconnection_id)`
- Trigger `invoice_items_protect_finalized` (BEFORE): `invoice_items_protect_finalized()`

adjustments

Credits and debits on an invoice (ADJ- numbers) with category and reason. Posted, never edited.

Column	Type	Null	Default	Key
id	uuid	no	`gen_random_uuid()`	PK
adjustment_number	text	no		
invoice_id	uuid	no		FK → invoices.id
subscriber_id	uuid	no		FK → invoices.subscriber_id
kind	text	no		
category	text	no		
amount_centavos	integer	no		
reason	text	no		
created_by_user_id	uuid	no		FK → users.id
created_at	timestamptz	no	`now()`	

- Unique `adjustments_adjustment_number_unique`: `UNIQUE (adjustment_number)`
- Check `adjustments_amount_positive`: `CHECK ((amount_centavos > 0))`
- Check `adjustments_category_valid`: `CHECK ((((kind = 'credit'::text) AND (category = ANY (ARRAY['discount'::text, 'service_outage'::text, 'billing_error'::text, 'goodwill'::text, 'other'::text]))) OR ((kind = 'debit'::text) AND (category = ANY (ARRAY['penalty'::text, 'reconnection_fee'::text, 'billing_error'::text, 'other'::text])))))`
- Check `adjustments_reason_present`: `CHECK ((length(TRIM(BOTH FROM reason)) >= 3))`
- Index `adjustments_invoice_idx`: `(invoice_id)`
- Index `adjustments_subscriber_idx`: `(subscriber_id, created_at)`
- Trigger `adjustments_no_update_delete` (BEFORE): `block_modification()`

ledger_entries

Every debit and credit of a subscriber: invoices, payments, reversals, adjustments, voids. Append-only; the balance is their sum.

Column	Type	Null	Default	Key
id	uuid	no	`gen_random_uuid()`	PK
seq	bigint	no		

Column	Type	Null	Default	Key
subscriber_id	uuid	no		FK → service_accounts.subscriber_id
service_account_id	uuid	yes		FK → service_accounts.id
entry_date	date	no		
entry_type	text	no		
reference	text	no		
description	text	no		
debit_centavos	integer	no	`0`	
credit_centavos	integer	no	`0`	
invoice_id	uuid	yes		FK → invoices.id
created_by_user_id	uuid	no		FK → users.id
created_at	timestamptz	no	`now()`	
payment_id	uuid	yes		FK → payments.id
adjustment_id	uuid	yes		FK → adjustments.id

- Check `ledger_entries_one_side`: `CHECK (((debit_centavos >= 0) AND (credit_centavos >= 0) AND ((debit_centavos = 0) <> (credit_centavos = 0))))`
- Check `ledger_entries_type_valid`: `CHECK ((entry_type = ANY (ARRAY['invoice'::text, 'invoice_void'::text, 'payment'::text, 'payment_reversal'::text, 'adjustment'::text])))`
- Index `ledger_entries_adjustment_idx`: `(adjustment_id)`
- Index `ledger_entries_invoice_idx`: `(invoice_id)`
- Index `ledger_entries_payment_idx`: `(payment_id)`
- Unique index `ledger_entries_seq_idx`: `(seq)`
- Index `ledger_entries_subscriber_idx`: `(subscriber_id, entry_date, seq)`
- Trigger `ledger_entries_no_update_delete` (BEFORE): `ledger_entries_block_modification()`

document_sequences

Counters for invoice, receipt, adjustment, batch and backup numbers, taken under a row lock so numbers never repeat or skip.

Column	Type	Null	Default	Key
name	text	no		PK
prefix	text	no		
next_value	bigint	no	`1`	

- Check `document_sequences_next_positive`: `CHECK ((next_value >= 1))`

Payments and GCash

Money received, how it was applied to invoices, reversals, and GCash submissions with their proof images.

payments

Every payment (RCPT- numbers): method, amount, allocated amount, status posted or reversed.

Column	Type	Null	Default	Key
id	uuid	no	`gen_random_uuid()`	PK

Column	Type	Null	Default	Key
receipt_number	text	no		
subscriber_id	uuid	no		FK → batch_accounts.subscriber_id
method	text	no		
amount_centavos	integer	no		
allocated_centavos	integer	no	`0`	
payment_date	date	no		
reference_number	text	yes		
notes	text	yes		
status	text	no	`'posted'::text`	
gcash_submission_id	uuid	yes		FK → gcash_submissions.id
received_by_user_id	uuid	no		FK → users.id
posted_at	timestamptz	no	`now()`	
collection_batch_id	uuid	yes		FK → batch_accounts.batch_id
collector_id	uuid	yes		FK → collection_batches.collector_id

- Check `payments_allocated_range`: `CHECK (((allocated_centavos >= 0) AND (allocated_centavos <= amount_centavos)))`
- Check `payments_amount_positive`: `CHECK ((amount_centavos > 0))`
- Check `payments_field_collection_shape`: `CHECK ((((collection_batch_id IS NULL) = (collector_id IS NULL)) AND ((collection_batch_id IS NULL) OR (method = ANY (ARRAY['cash'::text, 'cheque'::text])))))`
- Check `payments_gcash_shape`: `CHECK ((((method = 'gcash'::text) = (gcash_submission_id IS NOT NULL)) AND ((method <> 'gcash'::text) OR (reference_number IS NOT NULL))))`
- Unique `payments_gcash_submission_id_unique`: `UNIQUE (gcash_submission_id)`
- Check `payments_method_valid`: `CHECK ((method = ANY (ARRAY['cash'::text, 'gcash'::text, 'bank_transfer'::text, 'cheque'::text, 'other'::text])))`
- Unique `payments_receipt_number_unique`: `UNIQUE (receipt_number)`
- Check `payments_status_valid`: `CHECK ((status = ANY (ARRAY['posted'::text, 'reversed'::text])))`
- Index `payments_batch_idx`: `(collection_batch_id)`
- Index `payments_date_idx`: `(payment_date)`
- Unique index `payments_id_subscriber_idx`: `(id, subscriber_id)`
- Index `payments_receipt_number_trgm_idx`: `(receipt_number gin_trgm_ops)`
- Index `payments_reference_idx`: `(reference_number)`
- Index `payments_subscriber_idx`: `(subscriber_id, payment_date)`
- Trigger `payments_field_collection_in_progress` (BEFORE): `payments_field_collection_in_progress()`
- Trigger `payments_protect_posted` (BEFORE): `payments_protect_posted()`

payment_allocations

How much of a payment went to which invoice, oldest first unless chosen by hand. Append-only.

Column	Type	Null	Default	Key
id	uuid	no	`gen_random_uuid()`	PK
payment_id	uuid	no		FK → payments.id
invoice_id	uuid	no		FK → invoices.id
subscriber_id	uuid	no		FK → invoices.subscriber_id
amount_centavos	integer	no		

Column	Type	Null	Default	Key
source	text	no		
allocated_by_user_id	uuid	no		FK → users.id
allocated_at	timestamptz	no	`now()`	

- Check `payment_allocations_amount_positive`: `CHECK ((amount_centavos > 0))`
- Check `payment_allocations_source_valid`: `CHECK ((source = ANY (ARRAY['auto'::text, 'manual'::text, 'credit'::text])))`
- Index `payment_allocations_invoice_idx`: `(invoice_id)`
- Index `payment_allocations_payment_idx`: `(payment_id)`
- Trigger `payment_allocations_no_update_delete` (BEFORE): `block_modification()`

payment_reversals

The reversal of a payment, with who and why. The original stays. Append-only.

Column	Type	Null	Default	Key
id	uuid	no	`gen_random_uuid()`	PK
payment_id	uuid	no		FK → payments.id
reason	text	no		
reversed_by_user_id	uuid	no		FK → users.id
reversed_at	timestamptz	no	`now()`	

- Unique `payment_reversals_payment_id_unique`: `UNIQUE (payment_id)`
- Trigger `payment_reversals_no_update_delete` (BEFORE): `block_modification()`

gcash_submissions

GCash payments reported by customers: reference (unique), sender, amount, status pending, verified or rejected.

Column	Type	Null	Default	Key
id	uuid	no	`gen_random_uuid()`	PK
subscriber_id	uuid	no		FK → subscribers.id
reference_number	text	no		
sender_name	text	no		
sender_number	text	no		
amount_centavos	integer	no		
transaction_date	date	no		
notes	text	yes		
status	text	no	`'pending'::text`	
recorded_by_user_id	uuid	no		FK → users.id
recorded_at	timestamptz	no	`now()`	
reviewed_by_user_id	uuid	yes		FK → users.id
reviewed_at	timestamptz	yes		
rejection_reason	text	yes		

- Check `gcash_submissions_amount_positive`: `CHECK ((amount_centavos > 0))`

- Check `gcash_submissions_review_shape`: `CHECK ((((status = 'pending'::text) = (reviewed_at IS NULL)) AND ((reviewed_at IS NULL) = (reviewed_by_user_id IS NULL)) AND ((status = 'rejected'::text) = (rejection_reason IS NOT NULL))))`
- Check `gcash_submissions_status_valid`: `CHECK ((status = ANY (ARRAY['pending'::text, 'verified'::text, 'rejected'::text, 'reversed'::text])))`
- Unique index `gcash_submissions_live_reference_idx`: `(reference_number) WHERE (status = ANY (ARRAY['pending'::text, 'verified'::text]))`
- Index `gcash_submissions_reference_trgm_idx`: `(reference_number gin_trgm_ops)`
- Index `gcash_submissions_status_idx`: `(status, recorded_at)`
- Index `gcash_submissions_subscriber_idx`: `(subscriber_id)`
- Trigger `gcash_submissions_protect_reviewed` (BEFORE): `gcash_submissions_protect_reviewed()`

payment_proofs

Proof images of a GCash submission or a payment: stored name, type, size (5 MB max) and SHA-256. Append-only.

Column	Type	Null	Default	Key
id	uuid	no	`gen_random_uuid()`	PK
gcash_submission_id	uuid	yes		FK → gcash_submissions.id
payment_id	uuid	yes		FK → payments.id
storage_key	text	no		
original_filename	text	yes		
mime_type	text	no		
size_bytes	integer	no		
sha256	text	no		
uploaded_by_user_id	uuid	no		FK → users.id
uploaded_at	timestamptz	no	`now()`	

- Check `payment_proofs_mime_valid`: `CHECK ((mime_type = ANY (ARRAY['image/png'::text, 'image/jpeg'::text, 'image/webp'::text])))`
- Check `payment_proofs_one_owner`: `CHECK ((num_nonnulls(gcash_submission_id, payment_id) = 1))`
- Check `payment_proofs_sha256_shape`: `CHECK ((sha256 ~ '^[0-9a-f]{64}$'::text))`
- Check `payment_proofs_size_valid`: `CHECK (((size_bytes > 0) AND (size_bytes <= 5242880)))`
- Unique `payment_proofs_storage_key_unique`: `UNIQUE (storage_key)`
- Index `payment_proofs_payment_idx`: `(payment_id)`
- Index `payment_proofs_submission_idx`: `(gcash_submission_id)`
- Trigger `payment_proofs_no_update_delete` (BEFORE): `block_modification()`

Collections

Collection areas, collectors, their batches (route sheets), field collections and cash remittances.

collection_areas

Areas and routes subscribers belong to.

Column	Type	Null	Default	Key
id	uuid	no	`gen_random_uuid()`	PK
code	text	no		
name	text	no		

Column	Type	Null	Default	Key
description	text	yes		
is_active	boolean	no	`true`	
created_at	timestamptz	no	`now()`	
updated_at	timestamptz	no	`now()`	

- Unique `collection_areas_code_unique`: `UNIQUE (code)`
- Index `collection_areas_active_idx`: `(is_active)`

collectors

Field collectors; may be linked to a user account.

Column	Type	Null	Default	Key
id	uuid	no	`gen_random_uuid()`	PK
code	text	no		
full_name	text	no		
contact_number	text	yes		
user_id	uuid	yes		FK → users.id
is_active	boolean	no	`true`	
created_at	timestamptz	no	`now()`	
updated_at	timestamptz	no	`now()`	

- Unique `collectors_code_unique`: `UNIQUE (code)`
- Unique `collectors_user_id_unique`: `UNIQUE (user_id)`
- Index `collectors_active_idx`: `(is_active)`

collector_assignments

Which collector serves a subscriber, with effective dates; at most one open assignment per subscriber.

Column	Type	Null	Default	Key
id	uuid	no	`gen_random_uuid()`	PK
subscriber_id	uuid	no		FK → subscribers.id
collection_area_id	uuid	yes		FK → collection_areas.id
collector_id	uuid	yes		FK → collectors.id
effective_from	timestamptz	no	`now()`	
effective_to	timestamptz	yes		
assigned_by_user_id	uuid	no		FK → users.id
reason	text	yes		

- Check `collector_assignments_period_valid`: `CHECK (((effective_to IS NULL) OR (effective_to >= effective_from)))`
- Index `collector_assignments_area_idx`: `(collection_area_id)`
- Index `collector_assignments_collector_idx`: `(collector_id)`
- Unique index `collector_assignments_one_open_idx`: `(subscriber_id) WHERE (effective_to IS NULL)`
- Index `collector_assignments_subscriber_idx`: `(subscriber_id, effective_from)`

collection_batches

A collector's outing (CB- numbers): route sheet snapshot, lifecycle from open to closed, reconciliation figures and variance reason.

Column	Type	Null	Default	Key
id	uuid	no	`gen_random_uuid()`	PK
batch_number	text	no		
collector_id	uuid	no		FK → collectors.id
collection_area_id	uuid	yes		FK → collection_areas.id
collection_date	date	no		
status	text	no	`'open'::text`	
notes	text	yes		
created_by_user_id	uuid	no		FK → users.id
created_at	timestamptz	no	`now()`	
dispatched_by_user_id	uuid	yes		FK → users.id
dispatched_at	timestamptz	yes		
submitted_by_user_id	uuid	yes		FK → users.id
submitted_at	timestamptz	yes		
expected_cash_centavos	integer	yes		
remitted_cash_centavos	integer	yes		
difference_centavos	integer	yes		
variance_kind	text	yes		
variance_reason	text	yes		
reconciled_by_user_id	uuid	yes		FK → users.id
reconciled_at	timestamptz	yes		
closed_by_user_id	uuid	yes		FK → users.id
closed_at	timestamptz	yes		
cancelled_by_user_id	uuid	yes		FK → users.id
cancelled_at	timestamptz	yes		
cancel_reason	text	yes		

- Unique `collection_batches_batch_number_unique`: `UNIQUE (batch_number)`
- Check `collection_batches_cancel_shape`: `CHECK ((((status = 'cancelled'::text) = (cancelled_at IS NOT NULL)) AND (num_nonnulls(cancelled_at, cancelled_by_user_id, cancel_reason) = ANY (ARRAY[0, 3]))))`
- Check `collection_batches_close_shape`: `CHECK ((((status = 'closed'::text) = (closed_at IS NOT NULL)) AND ((closed_at IS NULL) = (closed_by_user_id IS NULL))))`
- Check `collection_batches_dispatch_shape`: `CHECK ((((dispatched_at IS NULL) = (dispatched_by_user_id IS NULL)) AND ((status = ANY (ARRAY['open'::text, 'cancelled'::text])) OR (dispatched_at IS NOT NULL))))`
- Check `collection_batches_reconcile_shape`: `CHECK ((((status = ANY (ARRAY['reconciled'::text, 'closed'::text])) = (reconciled_at IS NOT NULL)) AND (num_nonnulls(reconciled_at, reconciled_by_user_id, expected_cash_centavos, remitted_cash_centavos, difference_centavos, variance_kind) = ANY (ARRAY[0, 6]))))`
- Check `collection_batches_status_valid`: `CHECK ((status = ANY (ARRAY['open'::text, 'in_progress'::text, 'submitted'::text, 'remitted'::text, 'reconciled'::text, 'closed'::text, 'cancelled'::text])))`
- Check `collection_batches_submit_shape`: `CHECK ((((submitted_at IS NULL) = (submitted_by_user_id IS NULL)) AND ((status = ANY (ARRAY['open'::text, 'in_progress'::text, 'cancelled'::text])) = (submitted_at IS

NULL))))

- Check `collection_batches_variance_consistent`: `CHECK (((reconciled_at IS NULL) OR ((expected_cash_centavos >= 0) AND (remitted_cash_centavos >= 0) AND (difference_centavos = (remitted_cash_centavos - expected_cash_centavos)) AND (variance_kind = CASE WHEN (difference_centavos = 0) THEN 'balanced'::text WHEN (difference_centavos < 0) THEN 'shortage'::text ELSE 'overage'::text END) AND ((difference_centavos = 0) OR (length(TRIM(BOTH FROM COALESCE(variance_reason, ''::text))) >= 3)))))`
- Index `collection_batches_collector_idx`: `(collector_id, collection_date)`
- Unique index `collection_batches_id_collector_idx`: `(id, collector_id)`
- Index `collection_batches_status_idx`: `(status, collection_date)`
- Trigger `collection_batches_protect` (BEFORE): `collection_batches_protect()`

batch_accounts

The subscribers on a batch with the balance they owed when it was built.

Column	Type	Null	Default	Key
id	uuid	no	`gen_random_uuid()`	PK
batch_id	uuid	no		FK → collection_batches.id
subscriber_id	uuid	no		FK → subscribers.id
current_centavos	integer	no		
arrears_centavos	integer	no		
credit_centavos	integer	no		
total_due_centavos	integer	no		
added_by_user_id	uuid	no		FK → users.id
added_at	timestamptz	no	`now()`	

- Check `batch_accounts_amounts_valid`: `CHECK (((current_centavos >= 0) AND (arrears_centavos >= 0) AND (credit_centavos >= 0) AND (total_due_centavos = GREATEST(0, ((current_centavos + arrears_centavos) - credit_centavos)))))`
- Unique index `batch_accounts_batch_subscriber_idx`: `(batch_id, subscriber_id)`
- Index `batch_accounts_subscriber_idx`: `(subscriber_id)`
- Trigger `batch_accounts_protect` (BEFORE): `batch_accounts_protect()`

collector_remittances

Cash handed over by the collector; voided with a reason, never deleted.

Column	Type	Null	Default	Key
id	uuid	no	`gen_random_uuid()`	PK
batch_id	uuid	no		FK → collection_batches.id
collector_id	uuid	no		FK → collection_batches.collector_id
amount_centavos	integer	no		
notes	text	yes		
received_by_user_id	uuid	no		FK → users.id
received_at	timestamptz	no	`now()`	
voided_by_user_id	uuid	yes		FK → users.id
voided_at	timestamptz	yes		
void_reason	text	yes		

- Check `collector_remittances_amount_positive`: `CHECK ((amount_centavos > 0))`
- Check `collector_remittances_void_shape`: `CHECK (((num_nonnulls(voided_at, voided_by_user_id, void_reason) = ANY (ARRAY[0, 3])) AND ((void_reason IS NULL) OR (length(TRIM(BOTH FROM void_reason)) >= 3))))`
- Index `collector_remittances_batch_idx`: `(batch_id)`
- Index `collector_remittances_collector_idx`: `(collector_id, received_at)`
- Trigger `collector_remittances_protect` (BEFORE): `collector_remittances_protect()`

Service control

Suspensions for non-payment and the reconnection workflow, including the reconnection fee.

suspension_records

Each suspension: date, reason, who approved it.

Column	Type	Null	Default	Key
id	uuid	no	`gen_random_uuid()`	PK
service_account_id	uuid	no		FK → service_accounts.id
effective_date	date	no		
reason	text	no		
approved_by	text	no		
notes	text	yes		
past_due_invoice_count	integer	no		
past_due_centavos	integer	no		
suspended_by_user_id	uuid	no		FK → users.id
created_at	timestamptz	no	`now()`	

- Check `suspension_records_approved_by_present`: `CHECK ((length(TRIM(BOTH FROM approved_by)) > 0))`
- Check `suspension_records_reason_present`: `CHECK ((length(TRIM(BOTH FROM reason)) > 0))`
- Check `suspension_records_snapshot_nonneg`: `CHECK (((past_due_invoice_count >= 0) AND (past_due_centavos >= 0)))`
- Index `suspension_records_account_idx`: `(service_account_id, created_at)`
- Unique index `suspension_records_id_account_idx`: `(id, service_account_id)`
- Trigger `suspension_records_no_update_delete` (BEFORE): `block_modification()`

reconnection_records

Reconnection requests: technician, completion, fee or fee waiver.

Column	Type	Null	Default	Key
id	uuid	no	`gen_random_uuid()`	PK
service_account_id	uuid	no		FK → service_accounts.id
suspension_record_id	uuid	no		FK → suspension_records.id
status	text	no	`'requested'::text`	
request_date	date	no		
requested_by_user_id	uuid	no		FK → users.id
requested_at	timestamptz	no	`now()`	
fee_centavos	integer	no		

Column	Type	Null	Default	Key
fee_waived	boolean	no	`false`	
fee_waiver_reason	text	yes		
notes	text	yes		
technician_user_id	uuid	yes		FK → users.id
assigned_by_user_id	uuid	yes		FK → users.id
assigned_at	timestamptz	yes		
completion_date	date	yes		
completed_by_user_id	uuid	yes		FK → users.id
completed_at	timestamptz	yes		
cancel_reason	text	yes		
cancelled_by_user_id	uuid	yes		FK → users.id
cancelled_at	timestamptz	yes		

- Check `reconnection_records_assignment_shape`: `CHECK (((num_nonnulls(technician_user_id, assigned_by_user_id, assigned_at) = ANY (ARRAY[0, 3])) AND ((status <> 'assigned'::text) OR (technician_user_id IS NOT NULL))))`
- Check `reconnection_records_cancel_shape`: `CHECK ((((status = 'cancelled'::text) = (num_nonnulls(cancel_reason, cancelled_by_user_id, cancelled_at) = 3)) AND (num_nonnulls(cancel_reason, cancelled_by_user_id, cancelled_at) = ANY (ARRAY[0, 3]))))`
- Check `reconnection_records_completion_shape`: `CHECK ((((status = 'completed'::text) = (num_nonnulls(completion_date, completed_by_user_id, completed_at) = 3)) AND (num_nonnulls(completion_date, completed_by_user_id, completed_at) = ANY (ARRAY[0, 3])) AND ((completion_date IS NULL) OR (completion_date >= request_date))))`
- Check `reconnection_records_fee_nonneg`: `CHECK ((fee_centavos >= 0))`
- Check `reconnection_records_status_valid`: `CHECK ((status = ANY (ARRAY['requested'::text, 'assigned'::text, 'completed'::text, 'cancelled'::text])))`
- Check `reconnection_records_waiver_shape`: `CHECK ((fee_waived = (fee_waiver_reason IS NOT NULL)))`
- Index `reconnection_records_account_idx`: `(service_account_id, requested_at)`
- Unique index `reconnection_records_one_live_idx`: `(service_account_id) WHERE (status = ANY (ARRAY['requested'::text, 'assigned'::text]))`
- Unique index `reconnection_records_one_per_suspension_idx`: `(suspension_record_id) WHERE (status <> 'cancelled'::text)`
- Index `reconnection_records_status_idx`: `(status)`
- Index `reconnection_records_technician_idx`: `(technician_user_id)`
- Trigger `reconnection_records_protect` (BEFORE): `reconnection_records_protect()`

Settings, backups and integrity

Business settings, the backup history and the results of integrity checks.

application_settings

Business settings as key/value pairs (grace period, suspension threshold).

Column	Type	Null	Default	Key
key	text	no		PK
value	text	no		
updated_at	timestamptz	no	`now()`	

backup_history

Every backup (BK- numbers): kind, files, checksums, verification. Protected by a trigger: only verification fields change after completion.

Column	Type	Null	Default	Key
id	uuid	no	`gen_random_uuid()`	PK
backup_number	text	no		
kind	text	no		
status	text	no		
folder_name	text	no		
note	text	yes		
created_by_user_id	uuid	yes		FK → users.id
started_at	timestamptz	no	`now()`	
completed_at	timestamptz	yes		
database_bytes	bigint	yes		
proof_file_count	integer	yes		
proof_bytes	bigint	yes		
manifest_sha256	text	yes		
verification_status	text	yes		
verified_at	timestamptz	yes		
verification_message	text	yes		
error_message	text	yes		

- Unique `backup_history_backup_number_unique`: `UNIQUE (backup_number)`
- Check `backup_history_completed_shape`: `CHECK (((status <> 'completed'::text) OR ((completed_at IS NOT NULL) AND (manifest_sha256 IS NOT NULL) AND (database_bytes IS NOT NULL) AND (proof_file_count IS NOT NULL) AND (proof_bytes IS NOT NULL))))`
- Check `backup_history_failed_shape`: `CHECK (((status <> 'failed'::text) OR (error_message IS NOT NULL)))`
- Unique `backup_history_folder_name_unique`: `UNIQUE (folder_name)`
- Check `backup_history_in_progress_shape`: `CHECK (((status <> 'in_progress'::text) OR (completed_at IS NULL)))`
- Check `backup_history_kind_valid`: `CHECK ((kind = ANY (ARRAY['manual'::text, 'scheduled'::text, 'safety'::text])))`
- Check `backup_history_status_valid`: `CHECK ((status = ANY (ARRAY['in_progress'::text, 'completed'::text, 'failed'::text])))`
- Check `backup_history_verification_valid`: `CHECK (((verification_status IS NULL) OR (verification_status = ANY (ARRAY['passed'::text, 'failed'::text]))))`
- Unique index `backup_history_one_in_progress`: `(status) WHERE (status = 'in_progress'::text)`
- Index `backup_history_started_idx`: `(started_at)`
- Trigger `backup_history_protect` (BEFORE): `backup_history_protect()`

integrity_checks

Results of integrity checks (live, test restore, after restore). Append-only.

Column	Type	Null	Default	Key
id	uuid	no	`gen_random_uuid()`	PK

Column	Type	Null	Default	Key
scope	text	no		
backup_id	uuid	yes		FK → backup_history.id
passed	boolean	no		
results	jsonb	no		
run_by_user_id	uuid	yes		FK → users.id
ran_at	timestamptz	no	`now()`	

- Check `integrity_checks_backup_for_restores`: `CHECK (((scope = 'live'::text) OR (backup_id IS NOT NULL)))`
- Check `integrity_checks_scope_valid`: `CHECK ((scope = ANY (ARRAY['live'::text, 'test_restore'::text, 'after_restore'::text])))`
- Index `integrity_checks_ran_idx`: `(ran_at)`
- Trigger `integrity_checks_no_update_delete` (BEFORE): `block_modification()`

API Reference

Generated from the running route table: 121 routes. Every route except the public ones requires a session token (`Authorization: Bearer …`); the permissions listed are checked on the server on every request, against the database. Bodies, queries and path parameters are validated with the Zod schemas in `@bcis/shared`; errors are JSON `{ error, message }`, validation errors `{ error: "VALIDATION", issues }`.

/admin

Method	Path	Access
GET	`/admin/users`	`user.manage`
POST	`/admin/users`	`user.manage`
PATCH	`/admin/users/:id`	`user.manage`
POST	`/admin/users/:id/reset-password`	`user.manage`
POST	`/admin/users/:id/roles`	`user.manage`
POST	`/admin/users/:id/status`	`user.manage`
POST	`/admin/users/:id/unlock`	`user.manage`

/audit-logs

Method	Path	Access
GET	`/audit-logs`	`audit.view`
GET	`/audit-logs/filter-options`	`audit.view`

/auth

Method	Path	Access
POST	`/auth/login`	Public
POST	`/auth/logout`	Signed in (own account)
GET	`/auth/me`	Signed in (own account)
POST	`/auth/password`	Signed in (own account)

/backups

Method	Path	Access
GET	`/backups`	`backup.create`
POST	`/backups`	`backup.create`
POST	`/backups/:id/restore`	`backup.restore`
POST	`/backups/:id/test-restore`	`backup.create`
POST	`/backups/:id/verify`	`backup.create`

/billing

Method	Path	Access
POST	`/billing/discard-drafts`	`billing.generate`
POST	`/billing/finalize`	`billing.generate`
POST	`/billing/generate`	`billing.generate`
GET	`/billing/summary`	`billing.view`

/collection-areas

Method	Path	Access
GET	`/collection-areas`	`collection.view`
POST	`/collection-areas`	`collection.manage`
PATCH	`/collection-areas/:id`	`collection.manage`

/collection-batches

Method	Path	Access
GET	`/collection-batches`	`collection.view`
POST	`/collection-batches`	`collection.manage`
GET	`/collection-batches/:id`	`collection.view`
POST	`/collection-batches/:id/accounts`	`collection.manage`
DELETE	`/collection-batches/:id/accounts/:subscriberId`	`collection.manage`
POST	`/collection-batches/:id/cancel`	`collection.manage`
POST	`/collection-batches/:id/close`	`collection.close`
POST	`/collection-batches/:id/collections`	`collection.manage`
POST	`/collection-batches/:id/dispatch`	`collection.manage`
POST	`/collection-batches/:id/reconcile`	`collection.reconcile`
POST	`/collection-batches/:id/remittances`	`collection.manage`
POST	`/collection-batches/:id/remittances/:remittanceId/void`	`collection.manage`
POST	`/collection-batches/:id/submit`	`collection.manage`

/collection-reports

Method	Path	Access
GET	`/collection-reports/collectors`	`collection.view`
GET	`/collection-reports/collectors/export`	`collection.view` + `report.export`

/collectors

Method	Path	Access
GET	`/collectors`	`collection.view`

Method	Path	Access
POST	`/collectors`	`collection.manage`
PATCH	`/collectors/:id`	`collection.manage`
GET	`/collectors/available-users`	`collection.manage`

/dashboard

Method	Path	Access
GET	`/dashboard`	`report.view`

/gcash-proofs

Method	Path	Access
GET	`/gcash-proofs/:id`	`payment.view`

/gcash-submissions

Method	Path	Access
GET	`/gcash-submissions`	`payment.view`
POST	`/gcash-submissions`	`payment.create`
GET	`/gcash-submissions/:id`	`payment.view`
POST	`/gcash-submissions/:id/proofs`	`payment.create`
POST	`/gcash-submissions/:id/reject`	`gcash.verify`
POST	`/gcash-submissions/:id/verify`	`gcash.verify`

/health

Method	Path	Access
GET	`/health`	Public
GET	`/health/db`	Public

/integrity-checks

Method	Path	Access
GET	`/integrity-checks`	`backup.create`
POST	`/integrity-checks`	`backup.create`

/invoices

Method	Path	Access
GET	`/invoices`	`billing.view`
GET	`/invoices/:id`	`billing.view`

Method	Path	Access
POST	`/invoices/:id/adjustments`	`billing.adjust`
POST	`/invoices/:id/void`	`billing.void`

/payments

Method	Path	Access
GET	`/payments`	`payment.view`
POST	`/payments`	`payment.create`
GET	`/payments/:id`	`payment.view`
POST	`/payments/:id/reverse`	`payment.reverse`

/plans

Method	Path	Access
GET	`/plans`	`plan.view`
POST	`/plans`	`plan.manage`
PATCH	`/plans/:id`	`plan.manage`

/receivables

Method	Path	Access
GET	`/receivables`	`receivable.view`
GET	`/receivables/aging`	`receivable.view`
GET	`/receivables/aging/export`	`receivable.view` + `report.export`
GET	`/receivables/filter-options`	any of `receivable.view`, `suspension.manage`
GET	`/receivables/suspension-candidates`	`suspension.manage`

/reconnections

Method	Path	Access
GET	`/reconnections`	`suspension.manage`
GET	`/reconnections/:id`	`suspension.manage`
POST	`/reconnections/:id/assign`	`suspension.manage`
POST	`/reconnections/:id/cancel`	`suspension.manage`
POST	`/reconnections/:id/complete`	`suspension.manage`

/reports

Method	Path	Access
GET	`/reports/billing-vs-collection`	`report.view`
GET	`/reports/billing-vs-collection/export`	`report.view` + `report.export`

Method	Path	Access
GET	`/reports/collections`	`report.view`
GET	`/reports/collections/export`	`report.view` + `report.export`
GET	`/reports/exceptions`	`report.view`
GET	`/reports/exceptions/export`	`report.view` + `report.export`
GET	`/reports/revenue`	`report.view`
GET	`/reports/revenue/export`	`report.view` + `report.export`
GET	`/reports/subscribers`	`report.view`
GET	`/reports/subscribers/export`	`report.view` + `report.export`
GET	`/reports/user-activity`	`audit.view`
GET	`/reports/user-activity/export`	`audit.view` + `report.export`

/search

Method	Path	Access
GET	`/search`	`subscriber.view`

/service-accounts

Method	Path	Access
GET	`/service-accounts`	`service.view`
GET	`/service-accounts/:id`	`service.view`
PATCH	`/service-accounts/:id`	`service.manage`
POST	`/service-accounts/:id/collector`	`service.manage`
POST	`/service-accounts/:id/plan`	`service.manage`
POST	`/service-accounts/:id/rate`	`service.manage`
POST	`/service-accounts/:id/reconnections`	`suspension.manage`
GET	`/service-accounts/:id/service-control`	`service.view`
POST	`/service-accounts/:id/status`	`service.manage`
POST	`/service-accounts/:id/suspend`	`suspension.manage`

/settings

Method	Path	Access
GET	`/settings/receivables`	`settings.manage`
PATCH	`/settings/receivables`	`settings.manage`

/subscribers

Method	Path	Access
GET	`/subscribers`	`subscriber.view`

Method	Path	Access
POST	`/subscribers`	`subscriber.manage`
GET	`/subscribers/:id`	`subscriber.view`
PATCH	`/subscribers/:id`	`subscriber.manage`
POST	`/subscribers/:id/addresses`	`subscriber.manage`
PATCH	`/subscribers/:id/addresses/:addressId`	`subscriber.manage`
POST	`/subscribers/:id/assignment`	`subscriber.manage`
GET	`/subscribers/:id/audit`	`subscriber.view` + `audit.view`
GET	`/subscribers/:id/collections`	`subscriber.view` + `collection.view`
POST	`/subscribers/:id/contacts`	`subscriber.manage`
PATCH	`/subscribers/:id/contacts/:contactId`	`subscriber.manage`
GET	`/subscribers/:id/history`	`subscriber.view`
GET	`/subscribers/:id/ledger`	`billing.view`
GET	`/subscribers/:id/payment-context`	`payment.view`
POST	`/subscribers/:id/service-accounts`	`service.manage`
GET	`/subscribers/:id/statement`	`billing.view`
GET	`/subscribers/:id/statement/export`	`billing.view`
POST	`/subscribers/:id/status`	`subscriber.manage`

/technicians

Method	Path	Access
GET	`/technicians`	`suspension.manage`

Roles and permissions

Which permissions each role grants (seeded by `db:seed`, `apps/api/src/db/rbac.ts`).

Permission	Owner / Super Admin	Administrator	Cashier	Collection Supervisor	Accounting / Auditor	Technician	Read-only Viewer
`user.manage`	✓						
`audit.view`	✓	✓			✓		
`settings.manage`	✓						
`backup.create`	✓						
`backup.restore`	✓						
`subscriber.view`	✓	✓	✓	✓	✓		
`subscriber.manage`	✓	✓					
`plan.view`	✓	✓					
`plan.manage`	✓	✓					
`service.view`	✓	✓	✓			✓	

Permission	Owner / Super Admin	Administrator	Cashier	Collection Supervisor	Accounting / Auditor	Technician	Read-only Viewer
`service.manage`	✓	✓					
`billing.view`	✓	✓	✓	✓	✓		
`billing.generate`	✓	✓					
`billing.void`	✓	✓					
`billing.adjust`	✓	✓					
`payment.view`	✓	✓	✓	✓	✓		
`payment.create`	✓	✓	✓				
`payment.reverse`	✓	✓					
`payment.allocate`	✓	✓					
`gcash.verify`	✓	✓	✓				
`collection.view`	✓	✓		✓	✓		
`collection.manage`	✓	✓		✓			
`collection.reconcile`	✓			✓			
`collection.close`	✓			✓			
`receivable.view`	✓	✓	✓	✓	✓		✓
`suspension.manage`	✓	✓				✓	
`report.view`	✓	✓		✓	✓		✓
`report.export`	✓	✓		✓	✓		